TRƯỜNG ĐẠI HỌC
PHENIKAA KHOA CÔNG
NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP
Nhập môn trí tuệ nhân tạo

Phân loại Email: Spam và Ham

Nguyễn Quốc Hiếu	*23010170*	*23010170@st.phenikaa-uni.edu.vn*
Nguyễn Hoàng Hiệp	*23010451*	*23010451@st.phenikaa-uni.edu.vn*

Gvhd: Th.s Trần Đình Tân

Mục Lục

1. Giới thiệu. 1
1.1 Bối cảnh và động lực 1
1.2 Bài toán và mục tiêu 1
1.3 Input và output 2
2. Tìm hiểu giải pháp 2
2.1 Các công trình liên quan 2
2.2 Phương pháp đề xuất 3
3. Cơ sở lý thuyết: Kiến trúc Transformer và Cơ chế Attention 4
3.1 Cơ chế Attention 4
3.1.1 Bối cảnh: Hạn chế của các mô hình tuần tự (RNN/LSTM) 4
3.1.2 Ý tưởng cốt lõi 4
3.1.3 Scaled Dot-Product Attention 4
3.1.4 Self-Attention 5
3.1.5 Multi-Head Attention 6
3.2 Kiến trúc Transformer 7
3.2.1 Cấu trúc tổng thể Encoder-Decoder 8
3.2.2 Bên trong Encoder 8
3.2.3 Bên trong Decoder 9
4. Thiết kế và Triển khai Thực nghiệm 10
4.1 Quy trình thực nghiệm 10
4.2 Cấu trúc thư mục 11
5. Tiền xử lý dữ liệu 13
5.1 Phân tích và Khám phá Dữ liệu (Exploratory Data Analysis - EDA) 13
5.2 Tiền xử lý và làm sạch dữ liệu 14
6. Thiết kết mô hình 15
6.1 Mô hình 1: Mạng Nơ-ron Truyền thẳng (FFNN) với Sentence Embeddings 15
6.1.1 Kiến trúc và nguyên lý hoạt động 15
6.1.2 Tham số và siêu tham số 21
6.2 Mô hình 2: DistilBERT (Fine-tuning a Transformer model) 22
6.2.1 Kiến trúc và nguyên lý hoạt động 22
6.2.2 Tham số và siêu tham số 27
6.3 Mô hình 3: Hồi quy Logistic với Bag-of-Words 28
6.3.1 Kiến trúc và nguyên lý hoạt động 28
6.3.2 Tham số và siêu tham số 31
7. Huấn luyện và tối ưu hóa mô hình 32
7.1 Huấn luyện và tối ưu mô hình FFNN 32
7.1.1 Quy trình huấn luyện và tìm kiếm siêu tham số 32
7.1.2 Lưới siêu tham số 33

7.1.3 Kết quả và lựa chọn siêu tham số tối ưu 33
7.1.4 Tinh chỉnh (Fine – Turning) 34
7.2 Huấn luyện và tối ưu mô hình DistilBERT 35
7.2.1 Quy trình huấn luyện và tìm kiếm siêu tham số 35
7.2.2 Lưới siêu tham số 36
7.2.3 Kết quả và Lựa chọn bộ siêu tham số tối ưu 37
7.3 Huấn luyện và tối ưu mô hình Hồi quy Logistic 37
7.3.1 Quy trình huấn luyện và tìm kiếm siêu tham số 37
7.3.2 Lưới siêu tham số 38
7.3.3 Kết quả và Lựa chọn bộ siêu tham số tối ưu 38
8. Kết quả và Đánh giá 39
8.1 Lựa chọn độ đo đánh giá 39
8.2 Phân tích hiệu năng mô hình 40
9. Thảo luận 40
9.1 So sánh hiệu suất: 40
9.2 Phân tích lỗi và sự đánh đổi: 41
9.3 Khuyến nghị cuối cùng 42
10. Kết luận và Hướng phát triển 42
11. Phụ lục 43
11.1 Thuật toán Adam (Adaptive Moment Estimation) 43
11.1.1 Cơ sở lý thuyết và Mục tiêu 43
Nguyên lý toán học 43
11.2 Thuật toán AdamW (Adam with Decoupled Weight Decay) 44
11.2.1 Nguyên lý cải tiến của AdamW 44

Mục Lục Hình Ảnh

Hình 1. Sơ đồ minh họa cơ chế Scaled Dot-Product 5
Hình 2. Kiến trúc Scaled Dot-Product Attetion và Multi-Head Attention 7
Hình 3. Kiến trúc tổng thể Encoder-Decoder 8
Hình 4. Sơ đồ thể hiện cấu trúc Encoder-Decoder và cách cơ chế Multi-Head Attention được cấu thành từ Scaled Dot-Product Attention. 10
Hình 5. Dữ liệu ban đầu 14
Hình 6. Dữ liệu sau khi làm sạch 15
Hinh 7: Kiến trúc mô hình FFNN 21
Hình 8. Tương quan giữa distilBERT và BERT 24
Hình 9. Cấu trúc sử dụng mô hình DistilBERT của chương trình 27
Hình 10. Cấu trúc sử dụng mô hình LogisticRegression 30
Hình 11. Tổng hợp chỉ số đánh giá các mô hình 40
Hình 12. Tổng hợp confusion matrix của các mô hình 40

1. Giới thiệu.

1.1 Bối cảnh và động lực

Email là một trong những phương tiện giao tiếp điện tử không thể thiếu, đóng vai trò nền tảng cho cả công việc và đời sống cá nhân. Tuy nhiên, song hành với sự tiện lợi là một vấn đề ngày càng nhức nhối: thư rác (spam).

Thực tế, thư rác không chỉ là những quảng cáo gây phiền nhiễu. Mối đe dọa thực sự đến từ các chiến dịch lừa đảo (phishing) nhằm đánh cắp thông tin cá nhân, phát tán phần mềm độc hại (malware) để phá hoại hệ thống, hay các hình thức tấn công phi kỹ thuật khác. Chúng không chỉ gây lãng phí thời gian và tài nguyên hệ thống mà còn tiềm ẩn những rủi ro nghiêm trọng về an ninh và tài chính.

Với lưu lượng email lớn mỗi ngày, việc phân loại thủ công để tìm ra đâu là thư quan trọng (ham) và đâu là thư rác (spam) gần như là bất khả thi. Vì vậy, việc xây dựng một hệ thống lọc spam tự động, thông minh và hiệu quả đã trở thành một nhu cầu thiết thực, giúp bảo vệ người dùng và nâng cao năng suất làm việc.

1.2 Bài toán và mục tiêu

Bài toán đặt ra là xây dựng một mô hình học máy có khả năng tự động phân loại email thành hai nhóm: **"spam"** (thư rác) và **"ham"** (thư hợp lệ).

Điểm mấu chốt khi thiết kế hệ thống này là **"chi phí bất đối xứng"** của việc phân loại sai. Cụ thể:

- **Phân loại nhầm thư hợp lệ thành thư rác (Dương tính giả - False Positive)** là một lỗi cực kỳ nghiêm trọng. Bỏ lỡ một email công việc, một thông báo khẩn hay một cơ hội quan trọng có thể gây ra những thiệt hại lớn, làm giảm sút niềm tin của người dùng.

- **Bỏ lọt một thư rác vào hộp thư chính (Âm tính giả - False Negative)** tuy gây phiền toái và tiềm ẩn rủi ro nhưng mức độ nghiêm trọng thấp hơn đáng kể.

Vì lý do đó, **ưu tiên hàng đầu của mô hình là tối đa hóa Độ chính xác (Precision)**. Hệ thống cần được tinh chỉnh để có độ "thận trọng" cao, chỉ đánh dấu một email là spam khi các dấu hiệu thực sự rõ ràng và đáng tin cậy.

Các mục tiêu cụ thể của mô hình bao gồm:

- **Tối đa hóa Độ chính xác (Precision) cho lớp "Spam"**: Đây là ưu tiên hàng đầu. Mục tiêu là giảm thiểu đến mức gần như bằng không số lượng email quan trọng (ham) bị phân loại nhầm thành thư rác (spam). Việc bỏ lỡ một email công việc hay thông báo khẩn cấp có thể gây ra thiệt hại lớn.

- **Duy trì Recall cao cho lớp "Spam"**: Sau khi đã đảm bảo không lọc nhầm

thư quan trọng, mục tiêu tiếp theo là xác định được nhiều thư rác nhất có thể. Điều này giúp cải thiện trải nghiệm người dùng bằng cách giữ cho hộp thư chính sạch sẽ.

- **Đạt được Độ tin cậy tổng thể (Accuracy) cao**: Đảm bảo tỷ lệ phân loại đúng trên toàn bộ dữ liệu ở mức cao nhất có thể, phản ánh hiệu suất cân bằng của mô hình trên cả hai lớp.

1.3 Input và output

- **Input:** Mô hình sẽ tiếp nhận nội dung văn bản thô của một email, bao gồm cả tiêu đề (subject) và phần thân thư (body).

- **Đầu ra (Output):** Mô hình sẽ trả về một kết quả phân loại nhị phân: **"spam"** (nếu email được xác định là thư rác) hoặc **"ham"** (nếu là thư hợp lệ).

2. Tìm hiểu giải pháp

2.1 Các công trình liên quan

Lĩnh vực lọc thư rác đã trải qua nhiều giai đoạn phát triển công nghệ, mỗi giai đoạn đều mang lại những cải tiến đáng kể nhưng cũng đi kèm với những hạn chế riêng.

- **Giai đoạn 1: Lọc dựa trên Quy tắc (Rule-based Filtering)** Đây là phương pháp sơ khai nhất, hoạt động dựa trên một tập hợp các quy tắc tĩnh được định nghĩa trước (ví dụ: chặn email chứa từ khóa như "miễn phí", "trúng thưởng" hoặc từ các địa chỉ IP trong danh sách đen). Ưu điểm của phương pháp này là đơn giản và dễ triển khai. Tuy nhiên, nó thiếu tính linh hoạt, dễ bị qua mặt bởi các kỹ thuật lẩn tránh đơn giản (ví dụ: "m1ễn phí", "khuyếñ mãi") và đòi hỏi phải cập nhật bộ quy tắc thủ công một cách liên tục, tốn nhiều công sức.

- **Giai đoạn 2: Học máy Cổ điển (Classical Machine Learning)** Để khắc phục hạn chế của bộ lọc quy tắc, các thuật toán học máy cổ điển như Naive Bayes, Support Vector Machines (SVM), và Hồi quy Logistic đã được áp dụng. Các thuật toán này "học" các mẫu nhận diện thư rác từ dữ liệu đã được gán nhãn. Văn bản email thường được chuyển đổi thành vector số học thông qua các kỹ thuật như Bag-of-Words (BoW) hoặc TF-IDF. Mặc dù hiệu quả hơn đáng kể, hạn chế chính của các mô hình này là sự phụ thuộc vào tần suất từ, khiến chúng khó nắm bắt được ngữ nghĩa, mối quan hệ phức tạp và trật tự cú pháp của câu.

- **Giai đoạn 3: Học sâu và Kiến trúc Transformer (Deep Learning & Transformers)** Gần đây, các mô hình học sâu, đặc biệt là kiến trúc Transformer (BERT, RoBERTa, DistilBERT), đã tạo ra một bước đột phá trong lĩnh vực Xử lý Ngôn ngữ Tự nhiên (NLP). Thông qua cơ chế self-attention, các mô hình này có khả năng hiểu được ý nghĩa của một từ dựa

trên ngữ cảnh của toàn bộ câu. Điều này cho phép chúng nắm bắt các sắc thái tinh vi, phân biệt hiệu quả giữa thư rác và thư hợp lệ ngay cả khi chúng có nội dung tương tự nhau, qua đó đạt được hiệu suất vượt trội so với các phương pháp trước đây.

2.2 Phương pháp đề xuất

Dựa trên những phân tích về các công trình liên quan, nhóm nhận thấy các phương pháp học máy cổ điển tuy hiệu quả ở mức độ nhất định nhưng lại gặp hạn chế trong việc nắm bắt các mối quan hệ ngữ nghĩa phức tạp trong ngôn ngữ. Trong khi đó, các mô hình học sâu, đặc biệt là kiến trúc Transformer, đã tạo ra bước đột phá trong lĩnh vực Xử lý Ngôn ngữ Tự nhiên (NLP) nhờ khả năng hiểu sâu sắc ngữ cảnh của văn bản. Chính vì lẽ đó, **nhóm đã quyết định lựa chọn phương pháp tiếp cận dựa trên học sâu và Transformer làm hướng đi chủ đạo để giải quyết bài toán**, với niềm tin rằng hướng đi này sẽ mang lại hiệu suất vượt trội.

Để có một cái nhìn toàn diện và đưa ra lựa chọn tối ưu nhất, nhóm đã quyết định xây dựng và so sánh hiệu năng của ba mô hình tiêu biểu, đại diện cho cả hướng tiếp cận hiện đại và cổ điển:

- **Mô hình Mạng Nơ-ron Truyền thẳng (FFNN) kết hợp Sentence Embeddings**. Đây là một hướng tiếp cận lai, được lựa chọn nhằm cân bằng giữa hiệu suất và chi phí tính toán. Lý do nhóm xây dựng mô hình này là để tận dụng sức mạnh của các biểu diễn ngữ nghĩa hiện đại (từ Sentence Transformer) mà không cần đến chi phí tính toán khổng lồ của việc tinh chỉnh một mô hình Transformer lớn. Mô hình này được kỳ vọng sẽ có tốc độ xử lý nhanh và yêu cầu tài nguyên khiêm tốn.

- **Mô hình Tinh chỉnh (Fine-tuning) DistilBERT**. Đây là phương pháp tiếp cận hoàn toàn hiện đại, được xây dựng với kỳ vọng mang lại hiệu suất phân loại cao nhất. Nhóm lựa chọn DistilBERT vì đây là phiên bản gọn nhẹ, hiệu quả của **BERT (Bidirectional Encoder Representations from Transformers)** – một mô hình ngôn ngữ do Google phát triển, nổi tiếng với khả năng hiểu sâu sắc ngữ cảnh của từ trong câu. DistilBERT giữ lại tới 97% năng lực ngôn ngữ của bản gốc nhưng nhanh và nhỏ hơn đáng kể. Bằng cách tinh chỉnh toàn bộ mô hình trên dữ liệu của dự án, nhóm tin rằng mô hình có thể "học sâu" và thích ứng hoàn toàn với các đặc điểm ngôn ngữ đặc thù của thư rác, qua đó trở thành mô hình vượt trội nhất.

- **Mô hình Hồi quy Logistic với Bag-of-Words (Mô hình cơ sở)**. Mô hình này được xây dựng với vai trò là một baseline cổ điển. Việc so sánh kết quả của hai mô hình học sâu phức tạp hơn với baseline này sẽ giúp nhóm đánh giá một cách khách quan mức độ cải thiện mà chúng mang lại, đảm bảo tính khoa học và minh bạch cho quá trình nghiên cứu.

3. Cơ sở lý thuyết: Kiến trúc Transformer và Cơ chế Attention

3.1 Cơ chế Attention

3.1.1 Bối cảnh: Hạn chế của các mô hình tuần tự (RNN/LSTM)

Trước khi Transformer ra đời, các mô hình như Mạng Nơ-ron Hồi quy (RNN) và các biến thể tiên tiến hơn như LSTM, GRU là tiêu chuẩn vàng cho các bài toán xử lý chuỗi (sequence-to-sequence). Các mô hình này xử lý thông tin một cách tuần tự, từ trái sang phải

Tuy nhiên, kiến trúc này có một hạn chế cố hữu:

- **Vấn đề "thắt cổ chai" thông tin (Information Bottleneck):** Toàn bộ ý nghĩa của một câu đầu vào (dù dài hay ngắn) phải được nén vào một context vector duy nhất có kích thước cố định. Điều này khiến mô hình gặp khó khăn trong việc ghi nhớ các phụ thuộc xa (long-range dependencies) trong các chuỗi dài.

- **Khó khăn trong song song hóa:** Do tính chất tuần tự, việc tính toán trạng thái tại thời điểm t phụ thuộc vào kết quả tại $t-1$, gây cản trở việc tận dụng sức mạnh của các phần cứng hiện đại như GPU.

Cơ chế Attention ra đời để giải quyết chính những vấn đề này.

3.1.2 Ý tưởng cốt lõi

Thay vì buộc mô hình phải nhìn vào một vector ngữ cảnh duy nhất, cơ chế Attention cho phép mô hình "nhìn lại" toàn bộ chuỗi đầu vào tại mỗi bước giải mã (decoding). Nó học cách gán một attention score cho mỗi từ trong chuỗi đầu vào, tùy thuộc vào từ đang được tạo ra ở chuỗi đầu ra.

Nói một cách đơn giản, Attention cho phép mô-đun giải mã tập trung vào các phần có liên quan nhất của đầu vào để đưa ra dự đoán tiếp theo.

3.1.3 Scaled Dot-Product Attention

Scaled Dot-Product Attention là cơ chế tính toán cốt lõi cho Attention trong kiến trúc Transformer. Hoạt động của nó dựa trên ba thành phần chính được tạo ra từ chuỗi đầu vào:

1. Query (Q): Đại diện cho từ hoặc trạng thái hiện tại đang cần tìm thông tin. Trong ngữ cảnh dịch máy, đây có thể là trạng thái của bộ giải mã.

2. Key (K): Đại diện cho "nhãn" của các từ trong chuỗi đầu vào. Mỗi từ trong chuỗi nguồn sẽ có một vector Key tương ứng.

3. Value (V): Đại diện cho "nội dung" thực sự của các từ trong chuỗi đầu vào. Mỗi từ cũng có một vector Value.

Quá trình tính toán diễn ra như sau:

1. Scores (Tính điểm tương đồng): Lấy tích vô hướng của vector Query với tất cả các vector Key để đo lường mức độ "khớp" hay "liên quan".

$$\text{Scores} = Q \cdot K^T$$

2. Scale (Chia tỷ lệ): Chia các điểm số cho dk (căn bậc hai của số chiều của vector Key). Việc này giúp ổn định quá trình huấn luyện, tránh việc các giá trị quá lớn đẩy hàm softmax vào vùng có gradient gần bằng 0.

$$ScaledScores = \frac{Q \cdot K^T}{\sqrt{d_k}}$$

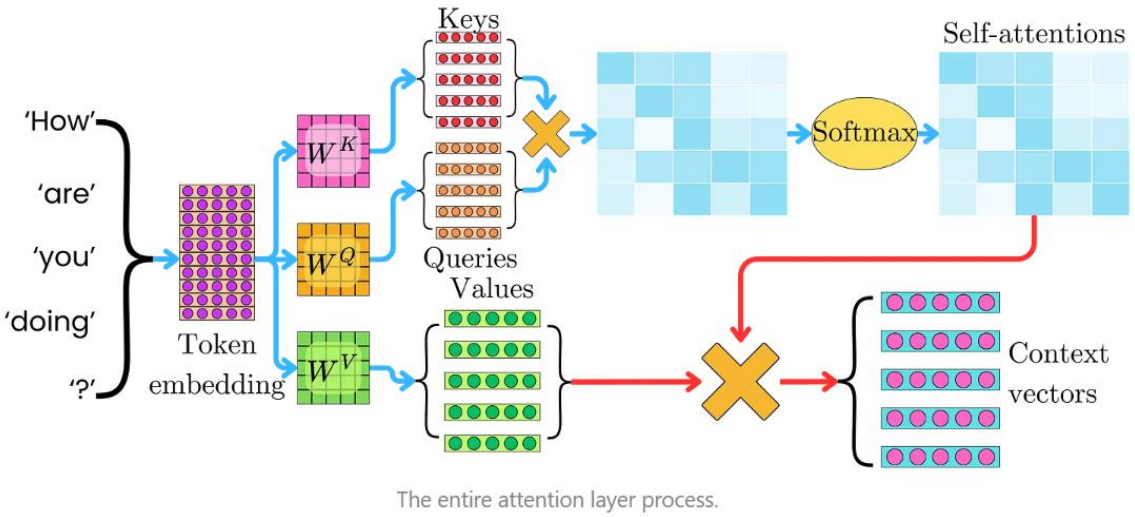
3. Tính Attention Weights: Áp dụng hàm softmax lên các điểm số đã chia tỷ lệ để tạo ra một phân phối xác suất. Các trọng số này cho biết mô hình nên "tập trung" bao nhiêu vào mỗi từ trong chuỗi đầu vào.

$$AttentionWeights = \text{softmax}(\frac{Q \cdot K^T}{\sqrt{d_k}})$$

4. Tính vector đầu ra: Lấy tổng có trọng số của các vector Value, sử dụng các trọng số attention đã tính ở bước trên.

$$Output = AttentionWeights \cdot V$$

Vector đầu ra cuối cùng là một sự pha trộn của các vector Value, được điều chỉnh bởi mức độ liên quan của chúng đối với Query hiện tại.



Hình 1. Sơ đồ minh họa cơ chế Scaled Dot-Product

3.1.4 Self-Attention

Self-Attention không phải là một cơ chế tính toán mới, mà là một **ứng dụng cụ thể** của Scaled Dot-Product Attention. Nó được gọi là "self" vì cơ chế này được áp dụng để tính toán mối quan hệ giữa các từ **bên trong cùng một chuỗi văn bản duy nhất**.

Điểm khác biệt cốt lõi và quan trọng nhất nằm ở nguồn gốc của ba thành phần chính: Query (Q), Key (K), và Value (V).

- Trong cơ chế attention chung (ví dụ: Encoder-Decoder Attention), Query có thể đến từ một chuỗi (chuỗi đích), trong khi Key và Value đến từ một chuỗi khác (chuỗi nguồn).

- Trong **Self-Attention**, cả ba vector Q, K, và V đều được tạo ra từ **cùng một nguồn duy nhất**: chuỗi đầu vào (cụ thể hơn là đầu ra của lớp trước đó).

3.1.5 Multi-Head Attention

Cơ chế này hoạt động dựa trên nguyên tắc chia để trị. Thay vì thực hiện một phép tính attention duy nhất trên toàn bộ vector đại diện cho một từ, Multi-Head Attention thực chất "cắt" hoặc chiếu vector đó thành nhiều phần nhỏ hơn, gọi là các "head".

Mỗi "head" có thể được xem như được giao nhiệm vụ tập trung vào một khía cạnh ngữ nghĩa cụ thể của từ. Ví dụ, nếu tưởng tượng một vector chứa đựng nhiều loại thông tin, thì "head" thứ nhất có thể được huấn luyện để chỉ tập trung vào thông tin liên quan đến "hành động", trong khi "head" thứ hai tập trung vào "tính chất" và "head" thứ ba tập trung vào "vị trí".

Sau khi "cắt" thông tin, quá trình attention sẽ diễn ra song song và độc lập bên trong từng nhóm "head". Tức là, tất cả các "head" về "hành động" của các từ trong câu sẽ tương tác với nhau, tương tự như vậy đối với các "head" về "tính chất". Quá trình này cho phép mô hình cùng lúc phân tích nhiều loại quan hệ khác nhau (ví dụ: quan hệ cú pháp, quan hệ ngữ nghĩa).

Cuối cùng, thông tin từ tất cả các "head" được kết hợp lại để tạo thành một vector biểu diễn cuối cùng, giàu thông tin hơn hẳn vì nó là kết quả tổng hợp từ nhiều "cách nhìn" hay nhiều khía cạnh phân tích khác nhau.

Cơ chế Multi-Head Attention vận hành theo một quy trình tính toán đa bước, được mô tả cụ thể như sau:

1. Tạo các Query, Key, và Value cho mỗi "Head": Đầu tiên, thay vì sử dụng một bộ ma trận trọng số duy nhất, Multi-Head Attention sử dụng h bộ ma trận trọng số khác nhau (một bộ cho mỗi "head"). Các embedding đầu vào được nhân với các ma trận này để tạo ra h bộ Query, Key và Value riêng biệt. Điều này cho phép mỗi "đầu" học cách chiếu thông tin đầu vào vào một không gian con khác nhau.

Với mỗi đầu $i \in \{1,...,h\}$, ta có:

- $Q_i = Q \cdot W_i^Q$
- $K_i = K \cdot W_i^K$
- $V_i = V \cdot W_i^v$ Trong đó W_i^Q, W_i^K, W_i^v là các ma trận trọng số riêng biệt cho đầu thứ i.

2. Tính toán Attention song song: Tiếp theo, cơ chế Scaled Dot-Product Attention được áp dụng một cách độc lập và song song cho từng bộ (Qi,Ki,Vi) đã tạo ở bước 1. Kết quả là **h** vector đầu ra, mỗi vector nắm bắt một loại thông tin liên quan khác nhau.

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)V_i$$

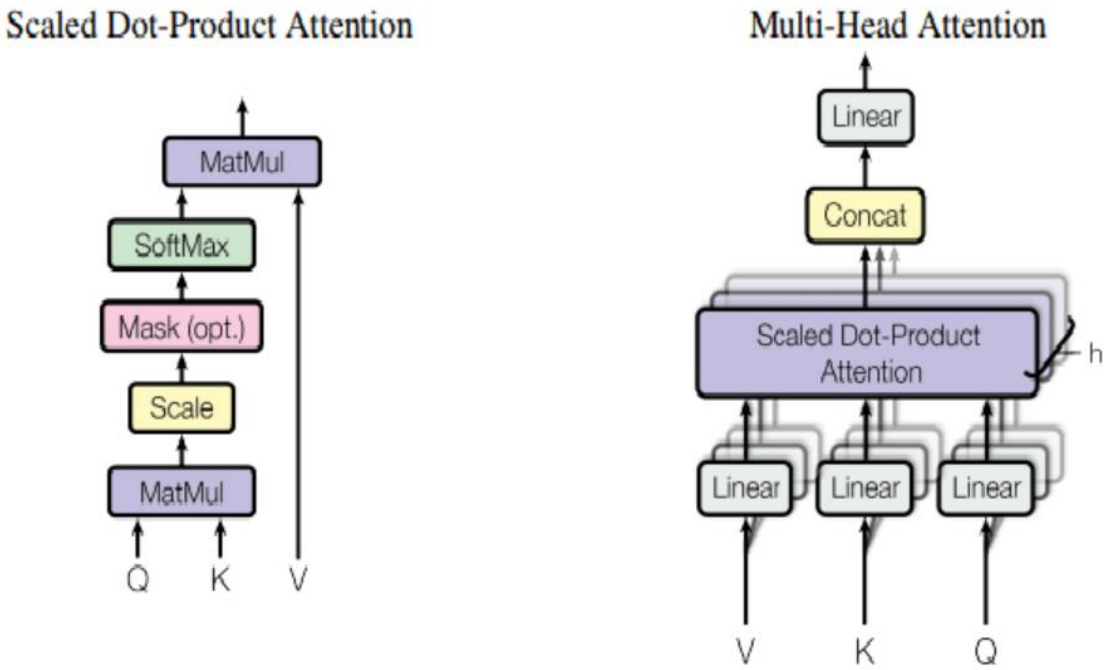
3. Nối (Concatenate) các kết quả: Các vector đầu ra từ **h** từ cơ chế attention được nối lại với nhau để tạo thành một ma trận duy nhất.

$$\text{Concat}(head_1, head_2, \ldots, head_h)$$

4. Chiếu (Project) lần cuối: Ma trận vừa được nối sẽ được nhân với một ma trận trọng số cuối cùng W^O, để chiếu nó trở lại không gian kích thước ban đầu của mô hình. Bước này cho phép mô hình tổng hợp và hòa trộn tất cả thông tin đã học được từ các "đầu" khác nhau.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, head_2, \ldots, head_h)W^O$$

Bằng cách này, Multi-Head Attention cho phép mô hình không chỉ quan tâm đến một từ khác, mà còn quan tâm đến từ đó ở nhiều khía cạnh và mối quan hệ khác nhau, làm cho biểu diễn cuối cùng trở nên phong phú và sâu sắc hơn rất nhiều.



Hình 2. Kiến trúc Scaled Dot-Product Attention và Multi-Head Attention

3.2 Kiến trúc Transformer

Transformer là một kiến trúc mô hình đột phá, được thiết kế để xử lý dữ liệu tuần tự, đặc biệt là văn bản, mà không cần sử dụng đến các mạng nơ-ron hồi quy (RNN).

Thay vào đó, nó hoàn toàn dựa vào cơ chế attention để nắm bắt các mối quan hệ phụ thuộc, dù ở gần hay xa, giữa các token trong dữ liệu đầu vào và đầu ra.

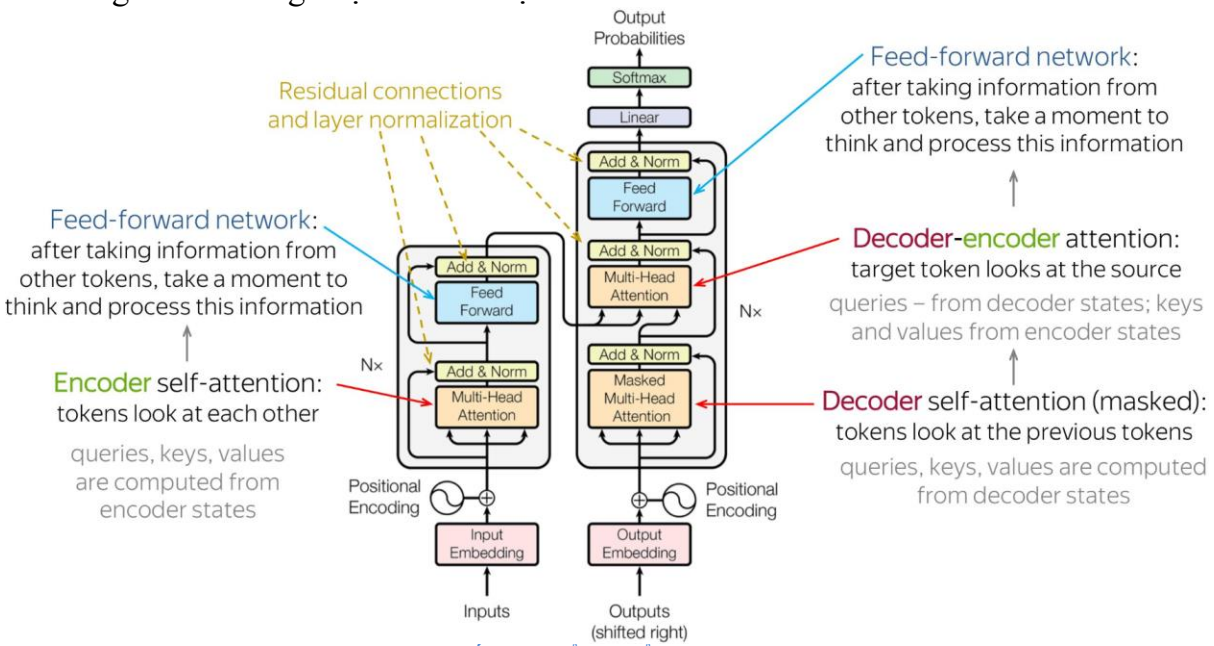
3.2.1 Cấu trúc tổng thể Encoder-Decoder

Kiến trúc Transformer duy trì một cấu trúc tổng thể bao gồm hai thành phần chính là Encoder (bộ mã hóa) và Decoder (bộ giải mã).

- **Encoder (Bộ mã hóa):** Nằm ở bên trái sơ đồ, nhiệm vụ của Encoder là tiếp nhận một chuỗi đầu vào (ví dụ: một câu trong ngôn ngữ nguồn) và xử lý nó để tạo ra một chuỗi các biểu diễn vector giàu ngữ cảnh (contextual representations). Toàn bộ khối Encoder là một chồng gồm N layers giống hệt nhau.

- **Decoder (Bộ giải mã):** Nằm ở bên phải sơ đồ, Decoder nhận đầu ra từ Encoder cùng với chuỗi đầu ra đã được tạo ra ở bước trước đó để dự đoán từ tiếp theo trong chuỗi đích. Tương tự như Encoder, Decoder cũng là một chồng gồm M lớp giống hệt nhau.

Cả hai chuỗi đầu vào của Encoder và Decoder đều được cộng với **Positional Encoding** trước khi đi vào các lớp xử lý. Điều này nhằm cung cấp cho mô hình thông tin về vị trí của các token trong chuỗi, vì kiến trúc chỉ dựa vào attention vốn không có khả năng nhận biết thứ tự.



Hình 3. Kiến trúc tổng thể Encoder-Decoder

3.2.2 Bên trong Encoder

Mỗi lớp trong N lớp của Encoder đều có cấu trúc giống nhau và bao gồm hai lớp con chính:

- **Multi-Head Self-Attention:** Đây là cơ chế thực hiện song song nhiều quá trình **Self-Attention**, trong đó mỗi head học cách tập trung vào các không gian con biểu diễn thông tin khác nhau của chuỗi đầu vào (ví dụ: một đầu

học quan hệ cú pháp, đầu khác học quan hệ ngữ nghĩa). Các đầu ra từ tất cả các đầu sau đó được kết hợp lại để tạo thành một biểu diễn cuối cùng duy nhất, phong phú và toàn diện hơn về ngữ cảnh.

- **Position-wise Feed-Forward Network:** Lớp con thứ hai là một mạng nơ-ron truyền thẳng. Sau khi lớp attention tổng hợp thông tin từ các token khác, mạng này có nhiệm vụ xử lý thông tin một cách độc lập tại từng vị trí để tạo ra một biểu diễn phức tạp hơn.

Sau mỗi lớp con, một kết nối phần dư (residual connection) và một lớp chuẩn hóa (layer normalization) được áp dụng. Thao tác này được biểu diễn bằng công thức:

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

trong đó *x* là đầu vào của lớp con và Sublayer(x) là đầu ra của lớp con đó (ví dụ: Multi-Head Attention hoặc Feed-Forward Network).

3.2.3 Bên trong Decoder

Mỗi lớp của Decoder có cấu trúc phức tạp hơn một chút, bao gồm ba lớp con chính:

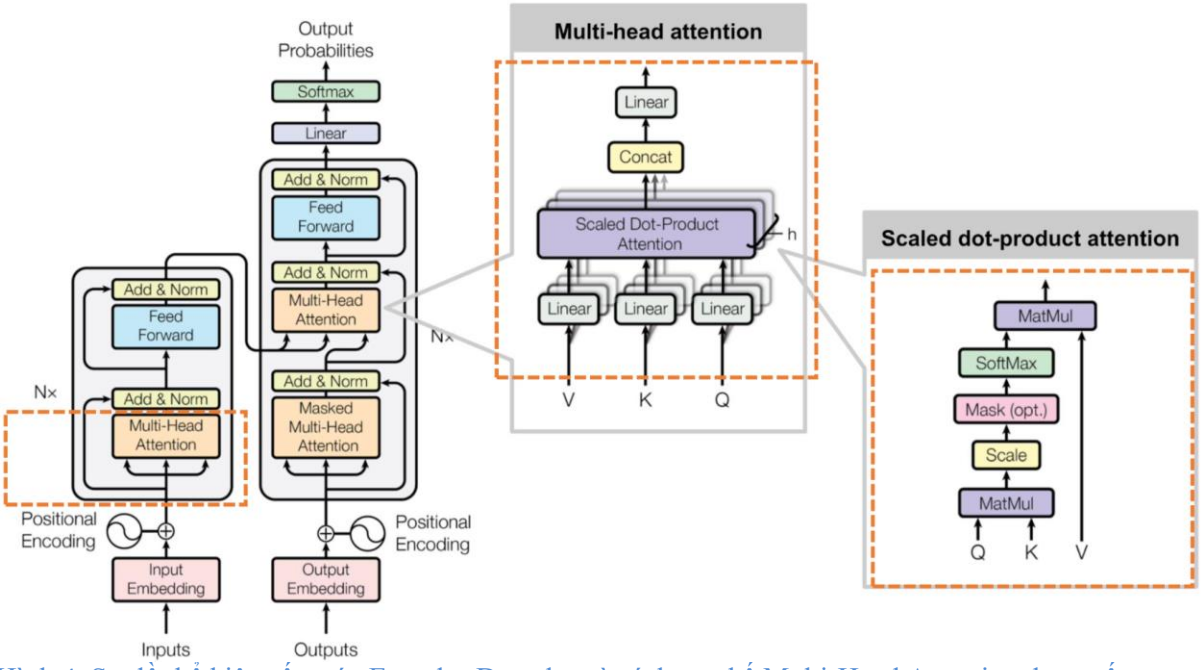
- **Masked Multi-Head Self-Attention:** Lớp này tương tự như lớp Multi-Head self-attention trong Encoder, nhưng có thêm một bước masking. Mục đích của mask là để đảm bảo rằng khi dự đoán một token ở vị trí i, mô hình chỉ có thể "nhìn" vào các token ở các vị trí trước đó (từ 1 đến $i-1$) trong chuỗi đầu ra, ngăn mô hình "gian lận" bằng cách nhìn vào các token tương lai.

- **Encoder-Decoder Attention:** Đây là nơi Decoder tương tác trực tiếp với Encoder. Trong lớp này, các vector Query (Q) đến từ lớp self-attention của Decoder, trong khi các vector Key (K) và Value (V) đến từ đầu ra của khối Encoder. Điều này cho phép mỗi token trong chuỗi đích (target) tập trung vào các phần có liên quan nhất của chuỗi nguồn.

- **Position-wise Feed-Forward Network:** Tương tự như trong Encoder, lớp này xử lý đầu ra từ lớp attention phía trước.

Tương tự Encoder, mỗi lớp con của Decoder cũng được theo sau bởi một kết nối phần dư và một lớp chuẩn hóa.

Cuối cùng, đầu ra của khối Decoder được đưa qua một lớp Linear và một hàm Softmax để tạo ra các xác suất cho từ vựng, từ đó chọn ra từ tiếp theo để dự đoán.



Hình 4. Sơ đồ thể hiện cấu trúc Encoder-Decoder và cách cơ chế Multi-Head Attention được cấu thành từ Scaled Dot-Product Attention.

4. Thiết kế và Triển khai Thực nghiệm

4.1 Quy trình thực nghiệm

Dự án được triển khai theo một quy trình có hệ thống, bao gồm các giai đoạn chính sau đây:

1. Khảo sát và Chuẩn bị Dữ liệu (Data Exploration & Preparation)

- Sử dụng bộ dữ liệu **spam_ham_dataset.csv** làm nguồn dữ liệu chính.
- Tiến hành **Phân tích và khám phá Dữ liệu (Exploratory Data Analysis - EDA)** để hiểu rõ các đặc trưng thống kê, phân phối độ dài email, sự cân bằng giữa hai lớp "spam" và "ham", và các từ khóa phổ biến.

2. Tiền xử lý Dữ liệu (Data Preprocessing)

- Thực hiện các bước làm sạch văn bản nhằm loại bỏ nhiễu và các thành phần không mang ngữ nghĩa như URL, địa chỉ email, thẻ HTML và các ký tự đặc biệt.
- Chuẩn hóa dữ liệu bằng cách chuyển toàn bộ văn bản sang chữ thường để đảm bảo tính nhất quán.

3. Triển khai Mô hình 1: FFNN & Sentence Embeddings

- **Vector hóa:** Chuyển đổi mỗi email đã được tiền xử lý thành một vector đặc trưng (embedding) 384 chiều bằng mô hình Sentence Transformer đã huấn luyện trước.

- **Huấn luyện:** Thiết kế và huấn luyện một Mạng Nơ-ron Truyền thẳng (FFNN) với đầu vào là các vector đặc trưng đã tạo.

- **Tối ưu hóa:** Sử dụng **GridSearchCV** để tinh chỉnh và tìm ra bộ siêu tham số tối ưu cho kiến trúc mạng FFNN.

- **Tinh chỉnh (Fine-tuning)**: Sau khi xác định được kiến trúc tối ưu, tiến hành một giai đoạn huấn luyện nâng cao bằng cách unfreeze các trọng số của mô hình all-MiniLM-L6-v2 và huấn luyện đồng thời với đầu phân loại, cho phép toàn bộ hệ thống thích ứng chuyên sâu với dữ liệu đặc thù của bài toán để tối đa hóa hiệu suất

4. Triển khai Mô hình 2: Tinh chỉnh DistilBERT

- **Mã hóa (Tokenization):** Xử lý văn bản email bằng tokenizer chuyên dụng của DistilBERT, chuyển đổi các câu thành định dạng đầu vào mà mô hình có thể hiểu được.

- **Tinh chỉnh (Fine-tuning):** Thiết lập và thực hiện quá trình huấn luyện lại (tinh chỉnh) toàn bộ mô hình DistilBERT trên bộ dữ liệu của bài toán để nó thích ứng với các đặc trưng ngôn ngữ riêng.

5. Triển khai Mô hình 3: Baseline Hồi quy Logistic

- Xây dựng một **pipeline** hoàn chỉnh, tích hợp hai bước: vector hóa văn bản bằng kỹ thuật **Bag-of-Words** và huấn luyện mô hình **Hồi quy Logistic**.

- Sử dụng **GridSearchCV** để tự động tìm ra bộ siêu tham số tốt nhất cho cả bước vector hóa và mô hình phân loại.

6. Đánh giá và So sánh Kết quả

- Đánh giá hiệu suất của cả ba mô hình trên test set độc lập.

- Sử dụng một bộ đầy đủ các độ đo: Accuracy, Precision, Recall và F1-Score.

- Phân tích sâu hơn thông qua **confusion matrix** để hiểu rõ về hành vi và các loại lỗi (Flase Negative, False Postive) của từng mô hình.

- Tổng hợp, so sánh kết quả để đưa ra kết luận về phương pháp hiệu quả nhất cho bài toán lọc thư rác.

4.2 Cấu trúc thư mục

Để đảm bảo tính tổ chức, tái sử dụng và dễ dàng quản lý, dự án được cấu trúc theo

một hệ thống thư mục rõ ràng. Cấu trúc này tuân theo các thông lệ tốt nhất trong các dự án học máy, tách biệt giữa dữ liệu, mã nguồn và các mô hình đã lưu.

Cấu trúc thư mục như sau:

```
Machine_Learning_Project/
│
├── checkpoints/
│   ├── ff_spam_classifier_model_spam_ham_dataset_cleaned.pth
│   └──distilbert_spam_YYYYMMDD_HHMMSS/
│           ├── distilbert_spam_classifier.pth
│           ├── tokenizer/
│           │       ├── special_tokens_map.json
│           │       ├── tokenizer_config.json
│           │       └── vocab.txt
│           └── training_config.json
│   └── logistic_regression_best.pkl
│
├── data/
│   ├── spam_ham_dataset.csv
│   └── embedding_spam_ham_datset_cleaned.pkl
│
├── distillbert_spam_classification.ipynb
├── FFNN_spam_classification.ipynb
└── logistic_spam_classification.ipynb
```

Chú thích: Thư mục checkpoints được tạo tự động khi chạy chương trình, đường link google drive chứa dữ liệu của thư mục checkpoints: https://drive.google.com/drive/folders/1fWxJpaGgUvly9ueaBNTiR8DmEV34Y61v?usp=drive_link

Giải thích cấu trúc:

- **Machine_Learning_Project/:** Thư mục gốc chứa toàn bộ dự án.
- **checkpoints/:** Thư mục này được dành riêng để lưu trữ các tệp trọng số của các mô hình đã được huấn luyện. Việc lưu lại các "checkpoint" này cho phép tái sử dụng mô hình để dự đoán trên dữ liệu mới mà không cần huấn luyện lại từ đầu.
    - **distilbert_spam_YYYYMMDD_HHMMSS/:** Mỗi lần huấn luyện và lưu lại, một thư mục con mới sẽ được tạo với tên chứa timestamp (ngày và giờ) để phân biệt các phiên bản.
- **data/:** Thư mục này chứa tất cả các dữ liệu và các cấu phần liên quan đến dữ liệu.

- spam_ham_dataset.csv: Tệp dữ liệu gốc.

- embedding_spam_ham_datset_cleaned.pkl: Tệp lưu các vector embedding đã được tính toán trước cho mô hình FFNN, giúp tăng tốc độ trong các lần chạy sau.

- ***.ipynb**: Các tệp notebook Jupyter chứa mã nguồn để xử lý dữ liệu, xây dựng, huấn luyện và đánh giá từng mô hình một cách riêng biệt, giúp mã nguồn trở nên module hóa và dễ theo dõi.

5. Tiền xử lý dữ liệu

5.1 Phân tích và Khám phá Dữ liệu (Exploratory Data Analysis - EDA)

Trong giai đoạn này, nhóm thực hiện phân tích dữ liệu khám phá trên bộ dữ liệu **spam_ham_dataset.csv** nhằm mục đích tìm hiểu các đặc trưng thống kê cơ bản, cấu trúc và sự phân phối của dữ liệu. Quá trình này là nền tảng để định hướng cho các bước làm sạch và xây dựng mô hình sau này.

Các thông tin chính được ghi nhận từ bộ dữ liệu như sau:

- **Nguồn và quy mô:** Bộ dữ liệu **spam_ham_dataset.csv** bao gồm **5171** email. Nguồn dữ liệu: https://www.kaggle.com/code/ayhampar/spam-ham-dataset/notebook

- **Cấu trúc:** Mỗi quan sát được mô tả qua các thuộc tính:
  - **text**: Nội dung văn bản của email.
  - **label**: Nhãn gốc, phân loại email là 'ham' (hợp lệ) hoặc 'spam' (thư rác).
  - **label_num**: Biểu diễn dạng số của nhãn (ví dụ: 0 cho 'ham', 1 cho 'spam').
  - **Unnamed: 0**: Một cột chỉ số được tạo tự động, không có giá trị về mặt ngữ nghĩa và sẽ được loại bỏ.

- **Phân phối các lớp (Class Distribution):** Phân tích cho thấy sự mất cân bằng dữ liệu giữa hai lớp:

  - **Ham (thư hợp lệ):** 3672 mẫu, chiếm **71.0%** tổng dữ liệu.
  - **Spam (thư rác):** 1499 mẫu, chiếm **29.0%** tổng dữ liệu.

	Unnamed: 0	label	text	label_num
0	605	ham	Subject: enron methanol ; meter # : 988291\nth...	0
1	2349	ham	Subject: hpl nom for january 9 , 2001\n(see a...	0
2	3624	ham	Subject: neon retreat\nho ho ho , we ' re arou...	0
3	4685	spam	Subject: photoshop , windows , office . cheap ...	1
4	2030	ham	Subject: re : indian springs\nthis deal is to ...	0

Hình 5. Dữ liệu ban đầu

Đánh giá về Mất cân bằng Dữ liệu và Chiến lược Xử lý:

Kết quả từ quá trình phân tích khám phá cho thấy một đặc tính nổi bật của bộ dữ liệu là sự mất cân bằng giữa các lớp. Cụ thể, lớp **ham** (thư hợp lệ) có số lượng mẫu cao hơn khoảng **2.45 lần** so với lớp **spam** (thư rác). Đây là một hiện tượng phổ biến trong các bộ dữ liệu thực tế và có khả năng gây ra sự thiên vị (bias) cho mô hình học máy. Nếu không được xử lý, mô hình sẽ có xu hướng dự đoán thiên về lớp đa số (**ham**), dẫn đến hiệu suất kém trong việc nhận diện lớp thiểu số (**spam**) - vốn là mục tiêu quan trọng của bài toán.

Để giải quyết vấn đề này, nhóm đề xuất một chiến lược hai phần:

- **Trong quá trình huấn luyện:** Sẽ áp dụng kỹ thuật điều chỉnh trọng số của hàm loss. Các tham số như **pos_weight** trong thư viện PyTorch hoặc **class_weight='balanced'** trong Scikit-learn sẽ được sử dụng để tăng "mức phạt" cho các lỗi dự đoán trên lớp thiểu số, qua đó buộc mô hình phải chú trọng hơn vào việc nhận diện chính xác lớp này.

- **Trong quá trình đánh giá:** Thay vì chỉ dựa vào Accuracy, vốn không phản ánh đúng hiệu năng trên dữ liệu mất cân bằng, nghiên cứu sẽ tập trung vào các độ đo toàn diện hơn như **Precision, Recall, và F1-Score**. Các chỉ số này cung cấp một cái nhìn sâu sắc và khách quan về khả năng của mô hình trong việc phân loại chính xác cả hai lớp, đặc biệt là khả năng nhận diện thư rác.

5.2 Tiền xử lý và làm sạch dữ liệu

Để đảm bảo chất lượng dữ liệu đầu vào và tối ưu hóa hiệu suất mô hình, nhóm đã thiết kế và triển khai một quy trình làm sạch dữ liệu bài bản. Quy trình này được thực hiện thông qua một hàm **clean_text** áp dụng tuần tự các bước sau:

- **Chuẩn hóa Tiêu đề:** Loại bỏ tiền tố "Subject:" xuất hiện phổ biến để đồng nhất hóa dữ liệu.

- **Loại bỏ Thông tin Nhiễu:** Các địa chỉ email và đường dẫn URL được xóa bỏ. Đây là các thông tin đặc thù, không có khả năng tổng quát hóa và có thể gây ra hiện tượng overfitting.

- **Trích xuất Nội dung Chữ:** Các ký tự đặc biệt (ví dụ: !, @, #, $) và chữ số được loại bỏ để giảm độ phức tạp của bộ từ vựng và giúp mô hình tập trung vào các đặc trưng ngôn ngữ quan trọng.

- **Chuẩn hóa Viết hoa:** Toàn bộ văn bản được chuyển sang dạng chữ thường để đảm bảo các biến thể của cùng một từ (ví dụ: "URGENT", "Urgent", "urgent") được xử lý như nhau.

- **Chuẩn hóa Khoảng trắng:** Loại bỏ các khoảng trắng thừa ở đầu, cuối và giữa các từ.

Quá trình làm sạch dẫn đến việc một số mẫu trở nên rỗng và đã được nhóm loại bỏ. Bộ dữ liệu cuối cùng sau khi xử lý còn lại **5153 mẫu** (3672 "ham", 1481 "spam") và sẵn sàng cho các giai đoạn huấn luyện và đánh giá tiếp theo.

	cleaned_text	label_num
0	enron methanol meter this is a follow up to th...	0
1	hpl nom for january see attached file hplnol x...	0
2	neon retreat ho ho ho we re around to that mos...	0
3	photoshop windows office cheap main trending a...	1
4	re indian springs this deal is to book the tec...	0

Hình 6. Dữ liệu sau khi làm sạch

6. Thiết kết mô hình

6.1 Mô hình 1: Mạng Nơ-ron Truyền thẳng (FFNN) với Sentence Embeddings

Mô hình này là một kiến trúc hai giai đoạn, kết hợp sức mạnh của transfer learning từ các mô hình ngôn ngữ lớn với một mạng nơ-ron tùy chỉnh hiệu quả.

6.1.1 Kiến trúc và nguyên lý hoạt động

Nguyên lý hoạt động của mô hình được chia thành hai giai đoạn chính: Trích xuất Đặc trưng và Phân loại.

➢ **Giai đoạn 1**: Trích xuất Đặc trưng (Feature Extraction)

Mỗi email trong tập dữ liệu, sau khi được làm sạch, sẽ được chuyển đổi thành một vector số thực có số chiều cố định. Thay vì sử dụng các phương pháp truyền thống

như Bag-of-Words, mô hình này tận dụng một mô hình Sentence Transformer đã được huấn luyện trước, cụ thể là all-MiniLM-L6-v2.

Nguyên lý của Sentence Transformer (all-MiniLM-L6-v2)

Mô hình all-MiniLM-L6-v2 được sử dụng trong giai đoạn trích xuất đặc trưng là một mô hình thuộc họ Sentence Transformers. Để hiểu rõ nguyên lý của nó, cần phải xem xét những hạn chế của các mô hình Transformer gốc như BERT và cách Sentence Transformers giải quyết chúng.

Hạn chế của BERT cho Tác vụ So sánh Câu

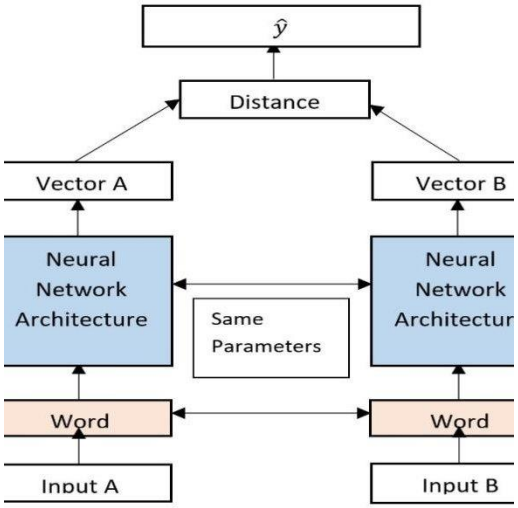
Một mô hình BERT tiêu chuẩn được tiền huấn luyện cho các tác vụ ở cấp độ từ (token-level) như Masked Language Modeling (Dự đoán từ bị che). Do đó, nó không được tối ưu hóa để tạo ra các vector embeddings cho toàn bộ câu một cách trực tiếp. Nếu chỉ lấy đầu ra của BERT và thực hiện một phép tổng hợp đơn giản như lấy trung bình các vector từ, kết quả thường là các vector biểu diễn câu (sentence embeddings) với chất lượng kém, không hiệu quả cho các tác vụ tìm kiếm ngữ nghĩa hoặc so sánh độ tương đồng.

Kiến trúc Mạng Siamese của Sentence Transformers

Sentence Transformers là một framework được thiết kế để khắc phục hạn chế trên bằng cách fine-tune các mô hình Transformer (như MiniLM) bằng cách sử dụng kiến trúc mạng **Siamese**. Kiến trúc này bao gồm:

- Hai mạng Transformer giống hệt nhau: Hai câu đầu vào, ví dụ câu A và câu B, được đưa qua hai mạng Transformer có cùng kiến trúc và chia sẻ cùng một bộ trọng số.

- Lớp tổng hợp (Pooling Layer): Đầu ra của mỗi mạng Transformer là một chuỗi các vector đặc trưng cho từng từ trong câu. Một lớp tổng hợp (thường là mean pooling) được áp dụng để chuyển đổi chuỗi vector này thành một vector duy nhất có kích thước cố định, đại diện cho toàn bộ câu. Quá trình này tạo ra các vector biểu diễn câu *u* cho câu A và *v* cho câu B.



Hình 3. Hình ảnh minh họa kiến trúc mạng Siamese

Hàm mục tiêu (Objective Function) trong quá trình Tinh chỉnh

Mục tiêu của quá trình tinh chỉnh là cấu trúc không gian embedding sao cho các câu có ngữ nghĩa tương đồng sẽ có vị trí gần nhau. Điều này đạt được bằng cách huấn luyện mô hình trên các tập dữ liệu lớn đã được gán nhãn cho các cặp câu, sử dụng các hàm mục tiêu chuyên biệt như:

- **Hàm mục tiêu Hồi quy (Regression Objective):** Được sử dụng cho các tác vụ như Semantic Textual Similarity (STS), nơi mục tiêu là dự đoán một điểm tương đồng. Hàm mất mát thường là Mean Squared Error giữa độ tương đồng cosine của hai vector câu và nhãn thực tế.

$$\hat{y} = \text{CosineSimilarity}(u, v) = \frac{u \cdot v}{||u|| \cdot ||v||} = \frac{\sum_{i=1}^{n} u_i v_i}{\sqrt{\sum_{i=1}^{n} u_1^2} \sqrt{\sum_{i=1}^{n} v_1^2}}$$

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

  Trong đó:
  - N là batch_size (số lượng samples).
  - n là số chiều của không gian vector.

- **Hàm mục tiêu Phân loại (Classification Objective):** Được sử dụng cho các tác vụ như Natural Language Inference (NLI). Các vector u và v được kết hợp và đưa qua một lớp phân loại để xác định mối quan hệ giữa hai câu (ví dụ: kéo theo, trung lập, mâu thuẫn). Hàm mất mát là Cross-Entropy.

$$\mathcal{L}_{CE} = -\sum_{c=1}^{C} y_c \log(\hat{p}_c)$$

  Trong đó:
  - C là tổng số lớp.

- y_c là giá trị thực tế (bằng 1 nếu mẫu thuộc lớp c, và 0 cho các lớp còn lại).
- $\hat{p}_c$ là xác suất dự đoán của mô hình cho lớp c.

Mô hình all-MiniLM-L6-v2

Mô hình **all-MiniLM-L6-v2** được lựa chọn là một phiên bản cụ thể, hiệu quả cao trong họ **Sentence Transformers**. Các đặc tính chính của nó được thể hiện qua tên gọi:

- MiniLM: Là một phiên bản được "chưng cất" (distilled) từ một mô hình Transformer lớn hơn, giúp nó nhỏ hơn, nhanh hơn đáng kể nhưng vẫn giữ được hiệu suất cao.

- L6: Mô hình có 6 lớp Transformer.

- all: Mô hình được tinh chỉnh trên một tập hợp rất lớn và đa dạng của nhiều bộ dữ liệu khác nhau, giúp nó trở thành một mô hình có general-purpose rất mạnh mẽ.

Kết quả của việc áp dụng mô hình này là mỗi email được chuyển đổi thành một vector 384 chiều, cô đọng lại ý nghĩa cốt lõi của nó, sẵn sàng để được sử dụng bởi mạng FFNN trong giai đoạn phân loại tiếp theo.

$$x = SentenceTransformer(E)$$

Trong đó:

- *x*: Là vector đặc trưng (feature vector) hay còn gọi là **embedding** đầu ra. Đây là một vector số thực 384 chiều, đại diện cho ý nghĩa ngữ nghĩa của toàn bộ email.
- *SentenceTransformer*: Là hàm biểu diễn cho mô hình **all-MiniLM-L6-v2**
- *E*: Là dữ liệu đầu vào của mô hình, đại diện cho nội dung văn bản thô của một email sau khi đã trải qua các bước làm sạch và tiền xử lý.

➢ **Giai đoạn 2**: Phân loại (Classification) bằng FFNN

Vector đặc trưng x sau đó được sử dụng làm đầu vào cho một Mạng Nơ-ron Truyền thẳng (FFNN). Quá trình lan truyền xuôi (forward pass) của mạng được mô tả như sau:

1. Đầu vào: Vector embedding x có kích thước 384. Vector này được giữ lại để sử dụng cho kết nối phần dư, gọi là residual.

$$x_{residual} = x$$

2. Khối Feed-Forward:

- Đầu vào x được đưa qua một khối mạng nơ-ron sâu bao gồm ba lớp Linear và hai hàm kích hoạt **Mish** xen kẽ.

$$\text{Mish}(x) = x \cdot \tanh(\ln(1 + e^x))$$

- Các lớp này biến đổi không gian đặc trưng, cho phép mô hình học các mối quan hệ phi tuyến phức tạp. Kiến trúc cụ thể là: Linear(384 -> hidden_dim) → Mish → Linear(hidden_dim -> hidden_dim) → Mish → Linear(hidden_dim -> 384).

$$h_1 = \text{Mish}(\text{Linear}_1(x))$$

$$h_2 = \text{Mish}(\text{Linear}_2(h_1))$$

$$\text{hff}_1 = \text{Linear}_3(h_2)$$

- Lớp tuyến tính cuối cùng trong khối này chiếu đầu ra trở lại không gian 384 chiều, đảm bảo nó có cùng kích thước với đầu vào residual để thực hiện bước tiếp theo.

3. Kết nối Phần dư (Residual Connection):

- Đầu ra của Khối Feed-Forward 1 được cộng trực tiếp với đầu vào ban đầu: out = out + residual.

- **Vai trò**: Kỹ thuật này rất quan trọng trong các mạng sâu. Nó tạo ra một "đường tắt" cho gradient trong quá trình lan truyền ngược, giúp giải quyết vấn đề vanishing gradient. Điều này cho phép mô hình được huấn luyện sâu hơn, hội tụ nhanh hơn và học các hàm phức tạp một cách hiệu quả hơn.

$$h_{res} = h_{ff1} + x_{residual}$$

4. Lớp Chuẩn hóa (Layer Normalization):

- Kết quả sau khi cộng kết nối phần dư được đưa qua một lớp LayerNorm.

$$h_{norm} = LayerNorm(h_{res})$$

- Vai trò: Lớp này giúp ổn định quá trình huấn luyện bằng cách chuẩn hóa các giá trị đầu ra của lớp trước đó trên các đặc trưng. Nó đảm bảo rằng các đầu vào cho lớp tiếp theo có phân phối ổn định (trung bình gần 0 và phương sai gần 1), giúp tăng tốc độ hội tụ.

5. Lớp Dropout:

- Một lớp Dropout được áp dụng. Trong quá trình huấn luyện, nó sẽ ngẫu nhiên "tắt" (đặt giá trị bằng 0) một tỷ lệ các nơ-ron.

$$h_{drop} = \text{Dropout}(h_{norm})$$

- **Vai trò**: Đây là một kỹ thuật regularization mạnh mẽ để giảm thiểu hiện tượng overfitting. Bằng cách ngăn các nơ-ron phụ thuộc quá nhiều vào nhau, Dropout buộc mô hình phải học các đặc trưng mạnh mẽ và có khả năng khái quát hóa tốt hơn trên dữ liệu mới.

6. Khối MLP:

- Vector đã được xử lý tiếp tục được đưa qua một khối MLP, bao gồm hai lớp tuyến tính và một hàm kích hoạt Mish: Linear(384 -> hidden_dim) → Mish → Linear(hidden_dim -> hidden_dim).

$$h_4 = \text{Mish}(\text{Linear}_4(h_{drop}))$$

$$h_{mlp2} = \text{Linear}_5(h_4)$$

- Khối này tiếp tục tinh chỉnh và học các đặc trưng ở mức độ trừu tượng cao hơn để chuẩn bị cho việc ra quyết định cuối cùng.

7. Lớp Output:

- Một lớp tuyến tính cuối cùng chuyển đổi đầu ra của khối MLP thành một giá trị vô hướng (kích thước 1), được gọi là logit. Logit này biểu thị điểm số thô mà mô hình gán cho việc email là "spam".

$$z = Linear_{out}(h_{mlp2})$$

8. Dự đoán và Hàm mất mát:

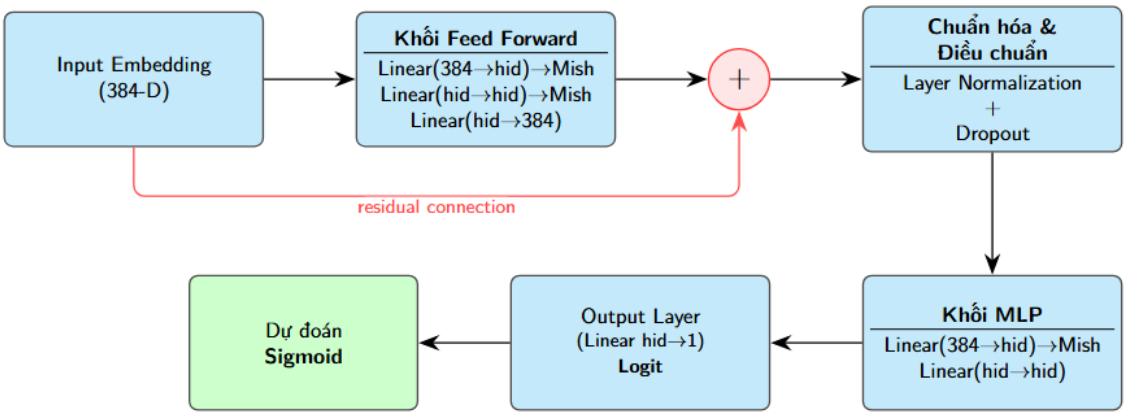
- Dự đoàn và hàm mất mát: Xác xuất dự đoán $\hat{y}$ (khả năng email là spam) được tính bằng các áp dụng hàm **sigmoid** cho logit z.

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Mô hình được huấn luyện bằng cách tối thiểu hóa hàm mất mát Binary Cross-Entropy with Logits BCEWithLogitsLoss. Hàm này kết hợp hàm sigmoid và BCELoss, giúp tăng tính ổn định về mặt số học. Để giải quyết vấn đề mất cân bằng dữ liệu, một trọng số pos_weight (p_w) được áp dụng cho lớp thiểu số (spam). Hàm mất mát cho một mẫu (y, z) được định nghĩa là:

$$L(\text{y}, \text{z}) = -[p_w \cdot \text{y} \cdot \log(\sigma(\text{z})) + (1 - \text{y}) \cdot \log(1 - \sigma(\text{z}))]$$

trong đó y ∈ {0,1} là nhãn thực tế và $p_w = \frac{số\ lượng\ mẫu\ ham}{số\ lượng\ mẫu\ spam}$



Hình 7: Kiến trúc mô hình FFNN

6.1.2 Tham số và siêu tham số

Hiệu suất của mô hình được quyết định bởi hai loại đại lượng: tham số và siêu tham số.

Tham số (Parameters) Đây là các giá trị mà mô hình học được trong quá trình huấn luyện thông qua thuật toán lan truyền ngược (backpropagation) và tối ưu hóa bằng gradient descent. Đối với mô hình này, các tham số bao gồm:

- Trọng số (weights) và độ lệch (biases) của tất cả các lớp tuyến tính (nn.Linear) trong kiến trúc mạng.
- Các tham số có thể học được (gamma và beta) của lớp chuẩn hóa (nn.LayerNorm).

Siêu tham số (Hyperparameters) Đây là các giá trị được cấu hình trước khi quá trình huấn luyện bắt đầu. Các giá trị chưa được chỉ định sẽ sử dụng gridSerachCV để tìm ra giá trị tốt ưu:

- **input_dim**: Kích thước của vector embedding đầu vào. Giá trị là 384, được xác định bởi mô hình Sentence Transformer.
- **output_dim**: Kích thước của lớp đầu ra. Giá trị là 1 cho bài toán phân loại nhị phân.
- **module__hidden_dim**: Số nơ-ron trong các lớp ẩn của cả hai khối feed-forward.
- **module__dropout_rate**: Tỷ lệ dropout để chống overfitting.

- **lr (Learning Rate)**: Quyết định mức độ cập nhật trọng số trong mỗi bước tối ưu hóa.

- **max_epochs**: Số lần toàn bộ tập dữ liệu huấn luyện được đưa qua mô hình.

- **Thuật toán tối ưu hóa (Optimizer):** thuật toán để cập nhật trọng số mạng.

- **Trọng số lớp dương (pos_weight)**: Khoảng 2.48 (tính được). Được sử dụng trong hàm mất mát để xử lý sự mất cân bằng giữa hai lớp "ham"và "spam".

6.2 Mô hình 2: DistilBERT (Fine-tuning a Transformer model)

6.2.1 Kiến trúc và nguyên lý hoạt động

Mô hình phân loại thứ hai của nhóm được xây dựng trên nền tảng DistilBERT, một kiến trúc Transformer được tối ưu về tốc độ và kích thước. Nhóm lựa chọn mô hình này vì nó cân bằng tốt giữa hiệu suất và yêu cầu tài nguyên (nhiều nhưng không vượt quá giới hạn của nhóm).

➢ **Nguyên lý và So sánh tương quan với BERT**

a. Nguyên lý chung:

DistilBERT là một phiên bản "chưng cất" (Distilled) của mô hình BERT, được tạo ra thông qua kỹ thuật **Knowledge Distillation**. Nguyên tắc của phương pháp này là huấn luyện một mô hình student – DistilBERT nhỏ hơn để bắt chước tri thức từ một mô hình teacher - BERT) lớn hơn.

Quá trình học này được điều khiển bởi một hàm mất mát (**loss function**) tổng hợp, bao gồm:

- **Distillation Loss ($L_{distill}$):** Buộc student phải sao chép toàn bộ phân phối xác suất đầu ra (soft labels) của teacher, thay vì chỉ học từ nhãn đúng (hard labels).

- **Student Loss ($L_{student}$):** Là hàm mất mát Cross-Entropy tiêu chuẩn, giúp student học trực tiếp từ nhãn đúng của dữ liệu để đảm bảo độ chính xác cho tác vụ.

- **Cosine Embedding Loss (L_{cos}):** Khuyến khích các biểu diễn vector (hidden states) của student trở nên tương đồng với của teacher.

Hàm mất mát cuối cùng là tổng có trọng số của ba thành phần trên:

$$L = \lambda_1 L_{distill} + \lambda_2 L_{student} + \lambda_3 L_{cos}$$

b. Đối sánh Hiệu năng với BERT-base

Những ưu điểm của DistilBERT được thể hiện rõ khi **đối sánh với mô hình**

BERT-base tiêu chuẩn:

- **Nhẹ hơn 40%:** Chỉ với 66 triệu tham số so với 110 triệu của BERT.
- **Nhanh hơn 60%:** Tăng tốc đáng kể cho cả quá trình huấn luyện và dự đoán.
- **Hiệu suất tương đương:** Duy trì được 97% hiệu năng của BERT trên các thang đo chuẩn.

Với những ưu điểm này, nhóm tin rằng DistilBERT là một lựa chọn phù hợp cho bài toán, kèm theo tốc độ phản hồi và hiệu quả triển khai là các yếu tố quan trọng bên cạnh độ chính xác khi xây dựng mô hình cho bài báo cáo.

➢ Kiến trúc DistilBERT

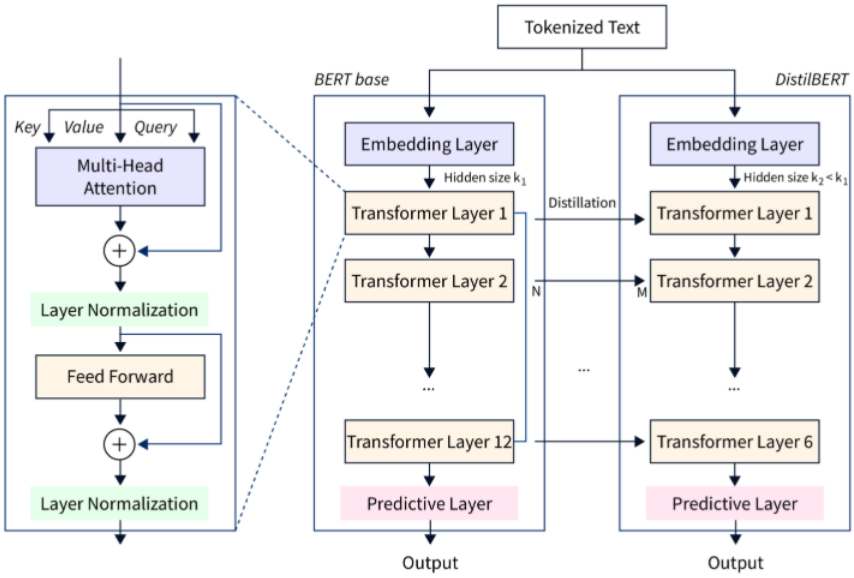
Kiến trúc tổng quan của DistilBERT được kế thừa từ BERT nhưng có một số sửa đổi quan trọng để giảm độ phức tạp:

- Số lượng lớp (Layers): Số lượng tầng mã hóa Transformer (Transformer Encoder layers) được giảm đi một nửa, từ 12 lớp trong BERT-base xuống còn 6 lớp.

- Loại bỏ các thành phần không cần thiết:
    - Token-type embeddings: Thành phần này trong BERT được dùng để phân biệt các câu khác nhau trong một cặp câu đầu vào. Nó đã bị loại bỏ.

    - Lớp Pooler: Lớp này trong BERT được dùng cho tác vụ dự đoán câu tiếp theo. Tác vụ này cũng được lược bỏ.

Kết quả là một kiến trúc gọn nhẹ hơn gồm 6 lớp Transformer encoder xếp chồng lên nhau.



Hình 8. Tương quan giữa distilBERT và BERT

➢ **Kiến trúc chương trình:**

Trong file code, mô hình phân loại không được xây dựng thủ công từ đầu. Thay vào đó, nó tận dụng lớp DistilBertForSequenceClassification từ thư viện transformers của Hugging Face. Đây là một ví dụ điển hình của transfer learning. Cấu trúc cụ thể bao gồm hai phần chính:

a. Mô hình DistilBERT cơ sở (Base Model):

- **Nguồn gốc:** đây là phần lõi của mô hình, được khởi tạo từ checkpoint đã tiền huấn luyện distilbert-base-uncased.

- **Đầu vào của mô hình**: Mô hình không xử lý trực tiếp văn bản thô. Trước tiên, email đầu vào được chuyển đổi thành các con số thông qua một tokenizer.

    - **Tokenization**: Chuỗi văn bản được tách thành các token (từ hoặc mảnh từ).

    - **Thêm token đặc biệt:**

        - **[CLS]** (Classification): Được thêm vào đầu mỗi chuỗi. Do cơ chế **Self-Attention** của Transformer, tại mỗi lớp, vector biểu diễn của [CLS] có khả năng "nhìn" và tương tác với tất cả các token khác trong câu. Mô hình sẽ học cách biến token **[CLS]** thành một "bản tóm tắt" hoàn hảo cho tác vụ phân loại.

        - **[SEP]** (Separator):Được thêm ở giữa các câu hoặc cuối chuỗi. **Phân tách các câu** hoặc đánh dấu **điểm kết thúc** của chuỗi đầu

vào.

Ví dụ, câu "check your email" sẽ trở thành [CLS] check your email [SEP]

- **Chuyển đổi thành ID**: Mỗi token được ánh xạ tới một ID duy nhất trong từ điển của tokenizer.

Kết quả là một chuỗi các ID số, $X_{input} = \text{Embedding}([\text{ID}_{[CLS]}, \text{ID}_{[token_1]}, \dots, \text{ID}_{[token_n]}, \text{ID}_{[SEP]}])$. sẵn sàng để đưa vào mô hình.

- **Kiến trúc: DistilBERT** bao gồm 6 lớp Transformer encoder. Nhiệm vụ chính của khối này là nhận chuỗi token đầu vào (sau khi đã được embedding) và tạo ra một biểu diễn ngữ cảnh sâu sắc cho mỗi token. Quá trình này được thực hiện thông qua cơ chế Multi-Head Self-Attention, cho phép mỗi từ "nhìn" vào các từ khác trong câu để hiểu ý nghĩa của nó trong ngữ cảnh.

  - Multi-Head Self-Attention

    $$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

    Với $\text{headi}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

  - Normalization

    $$X' = \text{LayerNorm}(X + \text{MultiHead}(X))$$

  - Khối FFNN

    $$\text{FFNNOutput} = \text{FFNN}(X')$$

    $$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

  - Normalization

    $$Y = \text{LayerNorm}(X' + \text{FFNOutput})$$

  Quá Trình trên lặp lại 6 lần

- **Đầu ra của khối cơ sở:** Một chuỗi các vector trạng thái ẩn (hidden state vectors), mỗi vector tương ứng với một token đầu vào.

$$\text{last_hidden_state} = [h_{[CLS]}, h_{[token_1]}, h_{[token_2]}, \dots, h_{[token_n]}, h_{[SEP]}]$$

b. Đầu phân loại (Classification Head):

Lớp DistilBertForSequenceClassification tự động thêm một "đầu phân loại" vào phía trên mô hình cơ sở. Đây là một mạng nơ-ron truyền thẳng đơn giản được thiết kế riêng cho tác vụ phân loại văn bản.

- **Đầu vào của đầu phân loại**: nó nhận vector biểu diễn của token đặc biệt [CLS] (vector đầu tiên trong chuỗi đầu ra của khối Transformer) làm đầu vào. Vector này được coi là biểu diễn tổng hợp cho toàn bộ email.

- **Đầu phân loại này bao gồm:**
    - Một lớp Pre-classifier (lớp tuyến tính và hàm kích hoạt ReLU) để biến đổi vector [CLS].

$$\mathrm{h}_{\mathrm{pre-classifier}} = \mathrm{ReLU}(\mathrm{W}_{pre} \cdot \mathrm{h}_{[CLS]} + \mathrm{b}_{pre})$$

    - Một lớp Dropout để ngăn ngừa hiện tượng overfitting.

    - Một lớp Classifier (lớp tuyến tính cuối cùng) chiếu đầu ra vào không gian có số chiều bằng số lượng nhãn. Vì đây là bài toán phân loại nhị phân (spam/ham), lớp này có 2 nơ-ron đầu ra.

$$\mathrm{Logits} = \mathrm{W}_{cls} \cdot h_{\mathrm{dropout}} + \mathrm{b}_{cls}$$

c. Logits và Hàm kích hoạt cuối cùng:

- Logits: Đầu ra của lớp Classifier là một cặp giá trị số thực, được gọi là logits. Các logits này biểu thị điểm số thô (raw scores) mà mô hình gán cho mỗi lớp (một logit cho "ham" và một logit cho "spam"). Chúng chưa phải là xác suất.

- Hàm kích hoạt trong Huấn luyện và Suy luận:
    - Trong quá trình huấn luyện: Các logits này được đưa trực tiếp vào hàm mất mát **nn.CrossEntropyLoss**. Hàm loss này c logits thành log-probabilities trước khi tính toán mất mát. Việc này mang lại tính ổn định số học cao hơn so với việc áp dụng Softmax rồi tính NLLLoss một cách riêng biệt.

$$\mathcal{L}_{CE}(z, c) = -\log\left(\frac{exp(z_c)}{\sum_{j=1}^{N} exp(z_i)}\right)$$

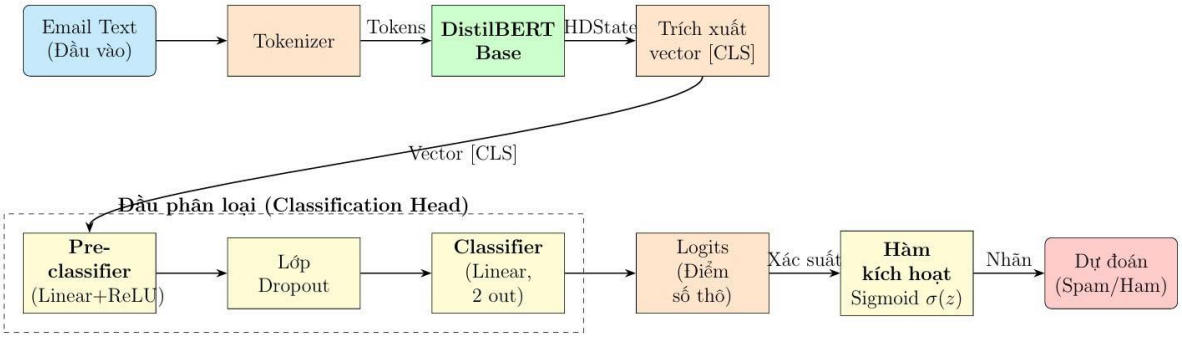
    - Trong quá trình suy luận (Inference/Prediction): Khi cần đưa ra dự đoán cụ thể hoặc tính toán độ tin cậy (confidence), chương trình đã áp dụng một cách tường minh hàm **Sigmoid** lên các logits đầu ra thay vì Softmax như bình thường **để các mô hình có sự tương đồng ở hàm kích hoạt đầu ra**.

$$\mathrm{P(spam)} = \sigma\left(\mathrm{logit}_{spam}\right) = \frac{1}{1 + e^{-logit_{spam}}}$$

Việc áp dụng Sigmoid cho mỗi logit trong bài toán phân loại nhị phân sẽ chuyển đổi mỗi điểm số thành một xác suất độc lập trong khoảng (0, 1). Quyết

định cuối cùng được đưa ra bằng cách so sánh xác suất của lớp "spam" với một ngưỡng

$$\text{Dự đoán } \begin{cases} spam & if\ P(spam) > 0.5 \\ ham & otherwise \end{cases}$$



Hình 9. Cấu trúc sử dụng mô hình DistilBERT của chương trình

6.2.2 Tham số và siêu tham số

Hiệu suất của mô hình được quyết định bởi hai loại đại lượng: tham số và siêu tham số.

Tham số (Parameters)

Đây là các giá trị mà mô hình học được trong quá trình huấn luyện thông qua thuật toán lan truyền ngược (backpropagation). Đối với mô hình này, các tham số bao gồm:

- Trọng số (weights) và độ lệch (biases): Của tất cả các lớp trong kiến trúc DistilBERT và lớp phân loại được thêm vào phía trên để phù hợp với bài toán.

Siêu tham số (Hyperparameters)

Đây là các giá trị được cấu hình trước khi quá trình huấn luyện bắt đầu. Các giá trị chưa được chỉ định sẽ sử dụng gridSerachCV để tìm ra giá trị tốt ưu:

- **max_length**: 256. Đây là số lượng token tối đa mà mô hình sẽ xem xét cho mỗi email. Nó đảm bảo tính đồng nhất về kích thước đầu vào bằng cách cắt bớt các email quá dài và thêm đệm (padding) vào các email quá ngắn.

- **Tốc độ học (Learning Rate)**: Một tốc độ học nhỏ là rất quan trọng khi tinh chỉnh các mô hình Transformer. Nó giúp điều chỉnh các trọng số một cách cẩn thận để thích ứng với tác vụ mới mà không làm mất đi kiến thức ngôn ngữ tổng quát đã học, tránh hiện tượng "quên lãng thảm khốc" (catastrophic forgetting).

- **Kích thước lô (Batch Size)**: Siêu tham số này xác định số lượng mẫu dữ liệu được đưa vào mô hình trong mỗi lần cập nhật trọng số.

- **Số chu kỳ (Epochs)**: Quá trình tinh chỉnh thường chỉ cần một số lượng nhỏ các chu kỳ. Huấn luyện trong 3 chu kỳ là đủ để mô hình hội tụ trên nhiệm vụ mới mà không bị overfitting vào bộ dữ liệu nhỏ.

- **Thuật toán tối ưu hóa (Optimizer)**: AdamW. Đây là một phiên bản cải tiến của thuật toán Adam, đặc biệt hiệu quả cho việc huấn luyện các mô hình Transformer

6.3 Mô hình 3: Hồi quy Logistic với Bag-of-Words

Nhóm xây dựng một mô hình Hồi quy Logistic với Bag-of-Words để làm baseline tham chiếu. Đây là một phương pháp tiếp cận cổ điển nhưng hiệu quả, cho phép đánh giá khách quan hơn hiệu năng của các mô hình Transformer phức tạp được trình bày ở các phần trước.

6.3.1 Kiến trúc và nguyên lý hoạt động

Mô hình hoạt động qua một quy trình hai giai đoạn được đóng gói một cách liền mạch bằng Pipeline của Scikit-learn, giúp tự động hóa và đảm bảo tính nhất quán.

➢ **Giai đoạn 1**: Vector hóa văn bản bằng Bag-of-Words (BoW)

- **Nguyên lý**: Các thuật toán học máy không thể xử lý văn bản thô trực tiếp. Do đó, bước đầu tiên là chuyển đổi mỗi email thành một vector số. Kỹ thuật Bag-of-Words (Túi từ) thực hiện điều này bằng cách biểu diễn văn bản dựa trên tần suất xuất hiện của các từ, bỏ qua hoàn toàn ngữ pháp, cấu trúc câu và thứ tự của từ. Nó "coi văn bản như một cái túi chứa các từ".

- **Kiến trúc**: Giai đoạn này được thực hiện bởi đối tượng CountVectorizer của thư viện Scikit-learn. Quá trình gồm hai bước nhỏ:

    - **Xây dựng từ vựng (Vocabulary):** CountVectorizer duyệt qua toàn bộ tập dữ liệu huấn luyện để xây dựng một bộ từ vựng, là danh sách tất cả các từ duy nhất xuất hiện.

$$V = \{\text{word}_1, \text{word}_2, \dots, \text{word}_n\} = \bigcup_{d_i \in D_{train}} \{\text{từ trong } d_i\}$$

Trong đó :

D: Tập hợp toàn bộ các email trong bộ dữ liệu

d_i: Một email thứ i trong tập dữ liệu D.

V: Bộ từ vựng (vocabulary), là tập hợp tất cả các từ duy nhất trong toàn bộ corpus.

- **Mã hóa (Encoding):** Với mỗi email, CountVectorizer tạo ra một vector. Mỗi chiều của vector tương ứng với một từ trong bộ từ vựng, và giá trị của chiều đó là số lần từ tương ứng xuất hiện trong email. Kết quả là một ma trận thưa (sparse matrix), trong đó mỗi hàng là một email và mỗi cột là một từ.

  - CountVectorizer:

$$x_{ij} = \text{Count}(w_j, d_i)$$

  - Vector đầy đủ cho email d_i:

$$\mathbf{x}_i = [x_{i1}, x_{i2}, \dots, x_{in}] = [\text{Count}(w_1, d_i), \text{Count}(w_2, d_i), \dots, \text{Count}(w_n, d_i)]$$

  - Spare matrix:

$$X = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_m \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \dots & x_{mn} \end{pmatrix}$$

Trong đó:
w_j: Từ thứ j trong bộ từ vựng V.
x_i: Vector đặc trưng của email di.
m: Tổng số email trong tập dữ liệu.
n: Kích thước của bộ từ vựng, n=|V|.

➢ **Giai đoạn 2**: Phân loại bằng Hồi quy Logistic

- **Nguyên lý**: Sau khi được vector hóa, các vector đặc trưng của email được đưa vào mô hình Hồi quy Logistic. Đây là một mô hình tuyến tính, học một tập hợp các trọng số (weights) cho mỗi đặc trưng (tức là mỗi từ trong từ vựng). Một trọng số dương lớn cho một từ có nghĩa là sự xuất hiện của từ đó làm tăng khả năng email là "spam", trong khi một trọng số âm lớn chỉ ra rằng nó có liên quan đến email "ham". Mô hình kết hợp các trọng số này và sử dụng hàm **Sigmoid** để đưa ra một xác suất dự đoán (từ 0 đến 1) cho biết email thuộc lớp "spam".

- **Kiến trúc**: Giai đoạn này được thực hiện bởi đối tượng LogisticRegression của Scikit-learn.

1. Mô hình tuyến tính

$$\text{z} = \mathbf{w^T x} + \text{b} = (\text{w}_1\text{x}_1 + \text{w}_2\text{x}_2 + \cdots + \text{w}_n\text{x}_n) + \text{b}$$

2. Hàm Sigmoid (Hàm Logistic)

$$\hat{y} = \text{P}(\text{y} = 1 \mid \mathbf{x}; \mathbf{w}, \text{b}) = \sigma(\mathbf{w^T x} + \text{b}) = \frac{1}{1 + e^{-(\mathbf{w^T x} + b)}}$$

3. Quyết định phân loại

$$y_{pred} = \begin{cases} 1 & if\ \hat{y} \geq 0.5(\text{Spam}) \\ 0 & if\ \hat{y} < 0.5(\text{Ham}) \end{cases}$$

4. Hàm mất mát: Hồi quy Logistic sử dụng hàm mất mát **Binary Cross-Entropy**

$$\mathcal{L}(\hat{y}, \mathrm{y}) = -[\mathrm{y}\log(\hat{y}) + (1-\mathrm{y})\log(1-\hat{y})]$$

5. Hàm chi phí: trung bình của các hàm mất mát trên tất cả m mẫu trong tập huấn luyện

$$J(\mathbf{w}, b) = -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)}\log(\hat{y}^{(i)}) + (1-y^{(i)})\log(1-\hat{y}^{(i)})]$$

6. Tối ưu hóa: Quá trình này được thực hiện bằng thuật toán tối ưu hóa Gradient Descent.

$$\mathbf{w} := \mathbf{w} - \alpha\frac{\partial J(\mathbf{w}, \mathrm{b})}{\partial \mathbf{w}}$$
$$\mathbf{b} := \mathbf{b} - \alpha\frac{\partial \mathrm{J}(\mathbf{w}, \mathrm{b})}{\partial \mathbf{b}}$$

- **Xây dựng pipeline**

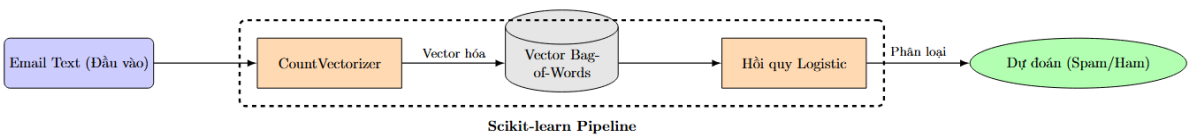
Toàn bộ quy trình từ vector hóa đến phân loại được gói gọn trong một đối tượng Pipeline, giúp đơn giản hóa quy trình làm việc, tránh rò rỉ dữ liệu (data leakage) khi thực hiện Cross-Validation, và giúp việc lưu trữ, tái sử dụng mô hình trở nên dễ dàng. Một Pipeline được xây dựng bao gồm hai bước chính:

- **vectorizer**: Sử dụng CountVectorizer để chuyển đổi văn bản email thành các vector Bag-of-Words.

$$\mathrm{d}_i \xrightarrow{BoW} \mathrm{x}_i$$

- **classifier**: Sử dụng LogisticRegression để thực hiện việc phân loại dựa trên các vector đã tạo. Việc sử dụng Pipeline đảm bảo rằng trong mỗi fold của quá trình Cross-Validation, bộ vector hóa (CountVectorizer) chỉ được "học" trên dữ liệu huấn luyện của fold đó, sau đó mới biến đổi cả dữ liệu huấn luyện và xác thực, qua đó ngăn chặn hoàn toàn hiện tượng rò rỉ dữ liệu.

$$\mathbf{x}_i \xrightarrow{\text{Logistic Regression}} \hat{y}_i \rightarrow \mathrm{y}_{pred,i}$$



Hình 10. Cấu trúc sử dụng mô hình LogisticRegression

6.3.2 Tham số và siêu tham số

Hiệu suất của mô hình được quyết định bởi hai loại đại lượng khác nhau:

Tham số (Parameters): Đây là các giá trị mà mô hình **tự học được** từ dữ liệu trong quá trình huấn luyện. Đối với mô hình này, tham số chính là vector trọng số θ. Mỗi phần tử trong θ tương ứng với một từ trong bộ từ vựng, thể hiện mức độ quan trọng của từ đó trong việc phân loại email.

Siêu tham số (Hyperparameters): Đây là các giá trị cấu hình được thiết lập **trước khi** quá trình huấn luyện bắt đầu. Các giá trị chưa được chỉ định sẽ sử dụng gridSerachCV để tìm ra giá trị tốt ưu:

- **vectorizer__max_features**: Số lượng từ phổ biến nhất được giữ lại trong bộ từ vựng của CountVectorizer. Việc giới hạn số lượng đặc trưng giúp giảm độ phức tạp tính toán, loại bỏ các từ hiếm (thường là nhiễu), và có thể giúp mô hình khái quát hóa hiệu quả hơn.

- **vectorizer__ngram_range**: Phạm vi của các n-gram (cụm từ) cần xem xét.
    - Giá trị (1, 1) chỉ xem xét các từ đơn (unigrams).
    - Giá trị (1, 2) xem xét cả từ đơn và các cặp từ liền kề (bigrams), giúp mô hình nắm bắt được một phần ngữ cảnh cục bộ (ví dụ: "miễn phí" có thể là một đặc trưng mạnh hơn là chỉ "miễn" và "phí" một cách riêng lẻ).

- **classifier__C**: Đây là siêu tham số regularization của Hồi quy Logistic. Nó là nghịch đảo của độ mạnh điều chuẩn.
    - Giá trị C nhỏ (ví dụ: 0.1) tương ứng với điều chuẩn mạnh hơn. Mô hình sẽ bị phạt nặng hơn khi có các trọng số lớn, dẫn đến một mô hình đơn giản hơn và giúp chống lại hiện tượng overfitting.
    - Giá trị C lớn (ví dụ: 10) tương ứng với điều chuẩn yếu hơn. Mô hình có nhiều "tự do" hơn để học các trọng số phức tạp, có thể phù hợp hơn với dữ liệu nhưng cũng có nguy cơ overfitting cao hơn.

- **classifier__class_weight**: (Giá trị được chỉ định: 'balanced') Đây là một siêu tham số quan trọng để xử lý vấn đề mất cân bằng dữ liệu. Khi được đặt là 'balanced', mô hình sẽ tự động điều chỉnh trọng số của các lớp tỷ lệ nghịch với tần suất xuất hiện của chúng. Cụ thể, lớp thiểu số ("spam") sẽ có trọng số cao hơn trong hàm chi phí, gia tăng mức phạt (penalty) cho các trường hợp phân loại sai, buộc mô hình phải chú ý hơn đến việc nhận diện đúng email spam.

7. Huấn luyện và tối ưu hóa mô hình

Quá trình huấn luyện được thực hiện riêng biệt cho từng mô hình, với các chiến lược tối ưu hóa và xử lý mất cân bằng dữ liệu phù hợp với từng kiến trúc.

7.1 Huấn luyện và tối ưu mô hình FFNN

Quá trình huấn luyện không chỉ đơn thuần là đưa dữ liệu vào mô hình mà còn là một giai đoạn tìm kiếm và tối ưu hóa để tìm ra bộ siêu tham số (hyperparameters) tốt nhất, giúp mô hình có khả năng tổng quát hóa cao trên dữ liệu mới. Đối với mô hình Mạng Nơ-ron Truyền thẳng (FFNN), nhóm đã áp dụng một phương pháp tiếp cận có hệ thống bằng cách kết hợp thư viện **Skorch** để tích hợp mô hình PyTorch vào quy trình làm việc của Scikit-learn, và sau đó sử dụng GridSearchCV để tự động hóa việc tìm kiếm siêu tham số.

7.1.1 Quy trình huấn luyện và tìm kiếm siêu tham số

Quy trình huấn luyện và tối ưu hóa được thực hiện theo các bước cụ thể sau:

1. Xử lý mất cân bằng dữ liệu: Như đã phân tích ở phần 3, bộ dữ liệu có sự mất cân bằng giữa hai lớp "ham" và "spam". Để mô hình không bị thiên vị, tức là có xu hướng dự đoán nghiêng về lớp đa số ("ham"), nhóm đã áp dụng một kỹ thuật xử lý trực tiếp trong hàm mất mát. Một trọng số **pos_weight** được tính toán bằng tỷ lệ giữa số lượng mẫu của lớp đa số và lớp thiểu số trên tập huấn luyện (số mẫu ham / số mẫu spam ≈ 2.48). Trọng số này sau đó được truyền vào hàm mất mát **BCEWithLogitsLoss** của PyTorch. Điều này làm cho các lỗi phân loại trên các mẫu "spam" (lớp thiểu số) bị phạt nặng hơn, buộc mô hình phải chú trọng hơn vào việc nhận diện đúng lớp này.

2. Tích hợp PyTorch với Scikit-learn: Để tận dụng công cụ GridSearchCV mạnh mẽ của Scikit-learn, nhóm đã sử dụng NeuralNetBinaryClassifier từ thư viện **Skorch**. Lớp này hoạt động như một "wrapper" (lớp bao bọc), giúp mô hình FeedForwardSpamClassifier được định nghĩa bằng PyTorch có thể hoạt động tương thích như một bộ phân loại tiêu chuẩn của Scikit-learn.

3. Thực hiện Grid Search: GridSearchCV được cấu hình để thực hiện một cuộc tìm kiếm toàn diện trên không gian các siêu tham số đã được định nghĩa trước. Với mỗi tổ hợp siêu tham số trong lưới:

- Mô hình được huấn luyện và đánh giá bằng kỹ thuật 3-fold cross-validation trên tập huấn luyện. Tập huấn luyện được chia thành 3 phần, mô hình được huấn luyện trên 2 phần và đánh giá trên phần còn lại, lặp lại 3 lần.

- Độ đo được sử dụng để đánh giá hiệu suất của mỗi tổ hợp là **F1-Score trung bình**. F1-score là một lựa chọn tốt cho dữ liệu mất cân bằng vì nó là trung bình điều hòa của Precision và Recall.

7.1.2 Lưới siêu tham số

Nhóm đã xác định một lưới các giá trị tiềm năng cho những siêu tham số quan trọng nhất ảnh hưởng đến hiệu suất của mô hình. GridSearchCV đã thử nghiệm tất cả các kết hợp có thể có từ lưới này:

Siêu tham số	Các giá trị được thử nghiệm	Vai trò
lr (Learning Rate)	[0.0001, 0.0005, 0.001]	Xác định mức độ cập nhật trọng số trong mỗi bước tối ưu hóa.
max_epochs	[20, 30]	Số lần tối đa mà toàn bộ tập dữ liệu được đưa qua mô hình.
optimizer	[torch.optim.Adam, torch.optim.AdamW]	Thuật toán được sử dụng để tối ưu hóa hàm mất mát.
module__hidden_dim	[INPUT_DIM//2, INPUT_DIM, INPUT_DIM * 2, INPUT_DIM*4]	Số nơ-ron trong lớp ẩn của kiến trúc mạng FFNN.
module__dropout_rate	[0.2, 0.3, 0.4]	Tỷ lệ dropout được sử dụng để điều chuẩn (regularization), giúp chống lại hiện tượng overfitting.

Chú thích: Tiền tố module__ được Skorch sử dụng để chỉ định rằng siêu tham số này sẽ được truyền vào mô-đun PyTorch (FeedForwardSpamClassifier) bên trong.

7.1.3 Kết quả và lựa chọn siêu tham số tối ưu

Sau khi hoàn tất quá trình tìm kiếm, GridSearchCV đã xác định được bộ siêu tham số mang lại điểm F1-Score trung bình cao nhất trên các tập xác thực.

Bộ siêu tham số tốt nhất được tìm thấy là:

- Tốc độ học (lr): 0.0005
- Số chu kỳ (max_epochs): 30
- Thuật toán tối ưu hóa (optimizer): torch.optim.AdamW
- Số chiều lớp ẩn (hidden_dim): 192 = INPUT_DIM/2
- Số phần trăm nơ-ron dropout: 0.2

Với bộ tham số này, mô hình đạt được điểm F1-score tốt nhất trên tập xác thực là 0.9687.

Cuối cùng, do tham số **refit=True** được bật trong GridSearchCV, một mô hình cuối cùng đã được tự động huấn luyện lại từ đầu trên toàn bộ tập dữ liệu huấn luyện (không phải cross-validation) bằng cách sử dụng bộ siêu tham số tối ưu đã tìm thấy ở trên. Chính mô hình đã được refit này (gs.best_estimator_) được lưu lại và sử dụng để đánh giá hiệu suất cuối cùng trên tập kiểm thử (test set).

7.1.4 Tinh chỉnh (Fine – Turning)

Sau khi đánh giá mô hình FFNN với đầu phân loại được huấn luyện trên các đặc trưng tĩnh, kết quả cho thấy hiệu suất chưa đạt mức tối ưu (F1-Score 0.97). Nguyên nhân chính được xác định là do các vector đặc trưng từ Sentence Transformer được giữ cố định, khiến mô hình ngôn ngữ không thể thích ứng với ngữ cảnh riêng của bài toán phân loại email.

Nhận thấy tiềm năng cải thiện còn lớn, nhóm đã chuyển sang chiến lược fine-tuning toàn bộ mô hình. Trong phương pháp này, các trọng số của mô hình ngôn ngữ **all-MiniLM-L6-v2** được unfreeze và huấn luyện đồng thời với đầu phân loại. Cách tiếp cận này cho phép toàn bộ hệ thống học các đặc trưng chuyên biệt và phù hợp nhất với dữ liệu, từ đó hướng đến việc đạt được hiệu suất phân loại cao hơn.

➢ **Kiến trúc Mô hình fine-tuning**

Một mô hình PyTorch mới, **FullSpamModel**, được xây dựng để bao bọc toàn bộ quy trình. Kiến trúc này gồm hai thành phần chính:

- **Lõi Sentence Transformer (all-MiniLM-L6-v2)**: Thành phần này chịu trách nhiệm chuyển đổi văn bản email thô (đã được tokenize) thành các vector đặc trưng (embeddings). Trong giai đoạn này, các trọng số của nó **không còn bị đóng băng** mà sẽ được cập nhật qua mỗi bước huấn luyện.

- **Đầu phân loại (Classifier Head)**: Nhóm tiếp tục sử dụng kiến trúc **FeedForwardSpamClassifier** đã được xác định là hiệu quả ở giai đoạn trước. Một điểm quan trọng là đầu phân loại này được khởi tạo với các trọng số đã được huấn luyện từ mô hình tốt nhất tìm thấy bởi **GridSearchCV**. Điều này giúp quá trình tinh chỉnh khởi đầu từ một điểm tối ưu, thay vì học lại từ đầu.

➢ **Quy trình Huấn luyện và Tối ưu hóa**

Quá trình tinh chỉnh được thực hiện thông qua một vòng lặp huấn luyện thủ công với các thiết lập sau:

- **Dữ liệu**: Dữ liệu văn bản thô được tokenize và đưa vào mô hình thông qua **DataLoader** của PyTorch với BATCH_SIZE là 256.

- **Tối ưu hóa**:
  - **Optimizer**: AdamW được lựa chọn, đây là thuật toán tối ưu hóa tiêu chuẩn và hiệu quả cho các mô hình Transformer.

  - **Learning Rate**: Một tốc độ học rất nhỏ (1e-5) được sử dụng. Điều này cực kỳ quan trọng trong fine-tuning, giúp mô hình điều chỉnh các trọng số một cách cẩn thận mà không phá vỡ các kiến thức ngôn ngữ tổng quát đã học.

  - **Scheduler**: CosineAnnealingLR được áp dụng để tự động điều chỉnh learning rate theo một chu kỳ cosine trong suốt quá trình huấn luyện, giúp mô hình hội tụ tốt hơn.

- **Epochs**: Mô hình được huấn luyện trong 10 epochs.

- **Lưu trữ mô hình**: Trong mỗi epoch, hiệu suất của mô hình được đánh giá trên validation set. Model state có **validation loss thấp nhất** sẽ được lưu lại làm mô hình cuối cùng. Kết quả cho thấy, mô hình tốt nhất đạt được ở epoch thứ 3 với validation loss là **0.1558**.

➢ **Kết quả Đánh giá Sau Tinh chỉnh**

Mô hình tốt nhất thu được từ quá trình fine-tuning được đánh giá lần cuối trên test set. Kết quả cho thấy sự cải thiện đáng kể so với mô hình chỉ sử dụng đặc trưng tĩnh:

- **Accuracy**: **0.9893** (tăng từ 0.9835)
- **Precision (Spam)**: 0.98 (không đổi)
- **Recall (Spam)**: **0.98** (tăng từ 0.96)
- **F1-Score (Spam)**: **0.98** (tăng từ 0.97)

Nhận xét: Việc tinh chỉnh toàn bộ mô hình đã giúp cải thiện rõ rệt các chỉ số quan trọng như Accuracy, Recall và F1-Score. Điều này khẳng định rằng việc cho phép mô hình ngôn ngữ tự thích ứng với dữ liệu đặc thù của bài toán là một phương pháp hiệu quả để đạt được hiệu suất cao nhất

7.2 Huấn luyện và tối ưu mô hình DistilBERT

Quá trình "tinh chỉnh" (fine-tuning) một mô hình Transformer như DistilBERT đòi hỏi một phương pháp tiếp cận cẩn thận để đạt được hiệu suất tối ưu mà không làm mất đi kiến thức ngôn ngữ tổng quát mà nó đã học được. Thay vì chỉ huấn luyện một lần, nhóm đã áp dụng cross-validation kết hợp với tìm kiếm siêu tham số để tìm ra cấu hình mạnh mẽ và ổn định nhất.

7.2.1 Quy trình huấn luyện và tìm kiếm siêu tham số

Không giống như các mô hình cổ điển, việc tinh chỉnh DistilBERT nhạy cảm hơn với các siêu tham số như learning rate và batch size. Do đó, một quy trình có hệ

thống đã được thực hiện:

1. Phân chia dữ liệu với Stratified K-Fold Cross-Validation: Để đánh giá một cách khách quan hiệu suất của các bộ siêu tham số khác nhau, nhóm đã sử dụng phương pháp **StratifiedKFold** với k=3 trên tập dữ liệu huấn luyện. Kỹ thuật này chia tập huấn luyện thành 3 fold có cùng tỷ lệ phân phối giữa hai lớp "ham" và "spam". Với mỗi bộ siêu tham số, mô hình sẽ được huấn luyện 3 lần, mỗi lần sử dụng 2 fold để huấn luyện và fold còn lại để đánh giá. Điều này giúp đảm bảo rằng hiệu suất được đánh giá là ổn định và không phụ thuộc vào một cách chia dữ liệu may rủi nào.

2. Vòng lặp huấn luyện: Trong mỗi fold, mô hình được huấn luyện với các thành phần chính sau:

- **Hàm mất mát (Loss Function):** nn.CrossEntropyLoss được sử dụng, đây là lựa chọn tiêu chuẩn cho các bài toán phân loại đa lớp (trong trường hợp này là 2 lớp). Hàm này kết hợp cả LogSoftmax và NLLLoss, giúp tăng tính ổn định về mặt số học.

- **Thuật toán tối ưu hóa (Optimizer):** AdamW được chọn. Đây là một biến thể của Adam, được cải tiến với kỹ thuật "Weight Decay" (giảm trọng số), đặc biệt hiệu quả cho việc huấn luyện các mô hình Transformer, giúp ngăn ngừa overfitting.

- **Bộ lập lịch Tốc độ học (Learning Rate Scheduler):** get_linear_schedule_with_warmup được áp dụng. Bộ lập lịch này giúp điều chỉnh tốc độ học một cách linh hoạt trong quá trình huấn luyện: tăng nhẹ ở giai đoạn đầu để ổn định, sau đó giảm dần tuyến tính, giúp mô hình hội tụ tốt hơn vào điểm tối ưu.

7.2.2 Lưới siêu tham số

Nhóm đã xác định một lưới các cấu hình siêu tham số để thử nghiệm thông qua quy trình **Cross-Validation**. Mục tiêu là tìm ra sự cân bằng tốt nhất giữa tốc độ huấn luyện và hiệu suất.

Siêu tham số	Các giá trị được thử nghiệm	Vai trò
learning_rate	[2e-5, 5e-5]	Tốc độ học nhỏ là rất quan trọng để tinh chỉnh, giúp mô hình thích ứng với dữ liệu mới mà không "quên" kiến thức cũ.
batch_size	[16, 32]	Số lượng mẫu dữ liệu được xử lý trong mỗi lần cập nhật trọng số.
epochs	[3, 4]	Số chu kỳ huấn luyện. Tinh chỉnh thường không

		cần quá nhiều epoch để tránh overfitting.

7.2.3 Kết quả và Lựa chọn bộ *siêu tham số tối ưu*

Sau khi thực hiện **Cross-Validation** cho tất cả các tổ hợp, hiệu suất của mỗi cấu hình được đánh giá dựa trên điểm F1-Score trung bình trên các fold xác thực.

Bộ siêu tham số tốt nhất được xác định là:

- learning_rate: 5e-5
- batch_size: 16
- epochs: 3

Cấu hình này đã đạt F1-Score trung bình là 0.9883 với độ lệch chuẩn rất thấp (±0.0010), chỉ ra rằng hiệu suất cao và rất ổn định.

Sau khi đã xác định bộ **hyperparameters** tối ưu thông qua **cross-validation**, mô hình **production** cuối cùng được huấn luyện lại trên toàn bộ dữ liệu (**train** và **test**). Quy trình này dựa trên nguyên tắc:

- **Validation**: Việc phân chia tập **train/test** ban đầu là bước xác thực bắt buộc, nhằm đảm bảo mô hình có khả năng tổng quát hóa và không bị **overfitting**.

- **Finalization**: Khi mô hình đã được xác thực, việc **fine-tune** trên toàn bộ dữ liệu giúp tối ưu hóa việc sử dụng tài nguyên thông tin, qua đó cải thiện nhẹ khả năng tổng quát hóa của mô hình cuối cùng.

Mô hình được lưu lại chính là kết quả của quá trình này, đạt **F1-Score** 0.9903.

7.3 Huấn luyện và tối ưu mô hình Hồi quy Logistic

Đối với mô hình Hồi quy Logistic, mặc dù là một phương pháp cổ điển, việc tối ưu hóa các siêu tham số vẫn đóng vai trò quyết định để đạt được hiệu suất cao nhất. Để thực hiện điều này một cách có hệ thống và tránh rò rỉ dữ liệu, Nhóm đã đóng gói toàn bộ quy trình, từ bước vector hóa văn bản đến huấn luyện bộ phân loại, vào một đối tượng Pipeline của Scikit-learn. Sau đó, công cụ GridSearchCV được sử dụng để tự động tìm kiếm tổ hợp siêu tham số tốt nhất.

7.3.1 Quy trình huấn luyện và tìm kiếm siêu tham số

Quy trình được thiết kế để tự động hóa và đảm bảo tính nhất quán:

1. Sử dụng Pipeline đã được xây dựng

2. Xử lý mất cân bằng dữ liệu: Tương tự như các mô hình trước, vấn đề mất cân bằng dữ liệu được giải quyết trực tiếp trong mô hình. Tham số **class_weight='balanced'** được thiết lập cho LogisticRegression. Cài đặt này sẽ tự

động điều chỉnh trọng số của các lớp trong hàm mất mát, gia tăng mức phạt (penalty) cho các trường hợp phân loại sai, buộc mô hình phải chú trọng hơn đến việc nhận diện đúng các email rác.

3. Tối ưu hóa bằng Grid Search: GridSearchCV được cấu hình với 5-fold cross-validation. Nó sẽ duyệt qua tất cả các tổ hợp siêu tham số được định nghĩa trong lưới, và với mỗi tổ hợp, mô hình sẽ được huấn luyện 5 lần. Độ đo được sử dụng để chọn ra tổ hợp tốt nhất là **F1-Score trung bình**.

7.3.2 Lưới siêu tham số

Lưới tìm kiếm bao gồm các siêu tham số cho cả hai bước trong Pipeline. Tên của mỗi tham số được đặt theo cú pháp tên_bước__tên_tham_số.

Siêu tham số	Các giá trị được thử nghiệm	Vai trò
vectorizer__max_features	[3000, 5000, 7000]	Giới hạn số lượng từ (đặc trưng) phổ biến nhất được giữ lại trong từ vựng, giúp giảm độ phức tạp và nhiễu.
vectorizer__ngram_range	[(1, 1), (1, 2)]	Xác định xem có nên chỉ sử dụng các từ đơn (unigrams) hay kết hợp cả các cụm từ đôi (bigrams) để nắm bắt ngữ cảnh cục bộ.
classifier__C	[0.1, 1, 10]	Tham số điều chuẩn (regularization) của Hồi quy Logistic. Giá trị C nhỏ tương ứng với điều chuẩn mạnh hơn, giúp chống overfitting.

7.3.3 Kết quả và Lựa chọn bộ *siêu tham số tối ưu*

Sau khi GridSearchCV hoàn thành việc thử nghiệm 18 tổ hợp khác nhau (3 x 2 x 3) trên 5 fold (tổng cộng 90 lần huấn luyện), bộ siêu tham số mang lại điểm **F1-Score trung bình** cao nhất trên các tập xác thực đã được tìm thấy.

Bộ siêu tham số tốt nhất được xác định là:

- classifier__C (Tham số regularization): 10 (tương ứng với regularization yếu hơn, chỉ ra rằng mô hình cần sự linh hoạt để phù hợp với dữ liệu).
- vectorizer__max_features (Số lượng đặc trưng): 5000
- vectorizer__ngram_range (Phạm vi N-gram): (1, 1) (chỉ sử dụng từ đơn cho hiệu quả tốt nhất).

Với cấu hình này, mô hình đạt được F1-score trung bình trên tập xác thực là 0.9750. Cuối cùng, một Pipeline hoàn chỉnh đã được tự động huấn luyện lại trên

toàn bộ tập dữ liệu huấn luyện với bộ tham số tối ưu này và được lưu lại để sử dụng cho việc đánh giá cuối cùng.

8. Kết quả và Đánh giá

8.1 Lựa chọn độ đo đánh giá

Việc lựa chọn độ đo phù hợp là cực kỳ quan trọng để đánh giá chính xác hiệu suất của mô hình, đặc biệt là khi mục tiêu của bài toán không chỉ là độ chính xác chung. Do ưu tiên hàng đầu là tối thiểu hóa việc bỏ lỡ email quan trọng (giảm thiểu False Positives), các độ đo sau đây được sử dụng để có một cái nhìn toàn diện :

- Accuracy (Độ chính xác tổng thể): Tỷ lệ phần trăm các dự đoán đúng trên tổng số mẫu.

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

Mặc dù phổ biến, độ đo này có thể gây hiểu lầm trên các tập dữ liệu mất cân bằng.

- Precision (Độ chính xác): Trong số tất cả các email được mô hình dự đoán là "Spam", có bao nhiêu email thực sự là "Spam". Đây là độ đo quan trọng nhất cho bài toán này. Precision cao có nghĩa là mô hình rất đáng tin cậy khi nó đánh dấu một email là spam.

$$\text{Precision} = \frac{TP}{TP+FP}$$

- Recall (Độ phủ hay Độ nhạy): Trong tổng số các email "Spam" thực tế có trong dữ liệu, mô hình đã tìm thấy (dự đoán đúng) được bao nhiêu email.

$$\text{Recall} = \frac{TP}{TP+FN}$$

- F1-Score: Là trung bình điều hòa (harmonic mean) của Precision và Recall. Đây là một độ đo tổng hợp tốt, đặc biệt hữu ích cho các tập dữ liệu mất cân bằng vì nó xem xét cả hai loại lỗi (False Positives và False Negatives).

$$\text{F1-Score} = 2 \cdot \frac{Precision \cdot Recall}{Precision+Recall}$$

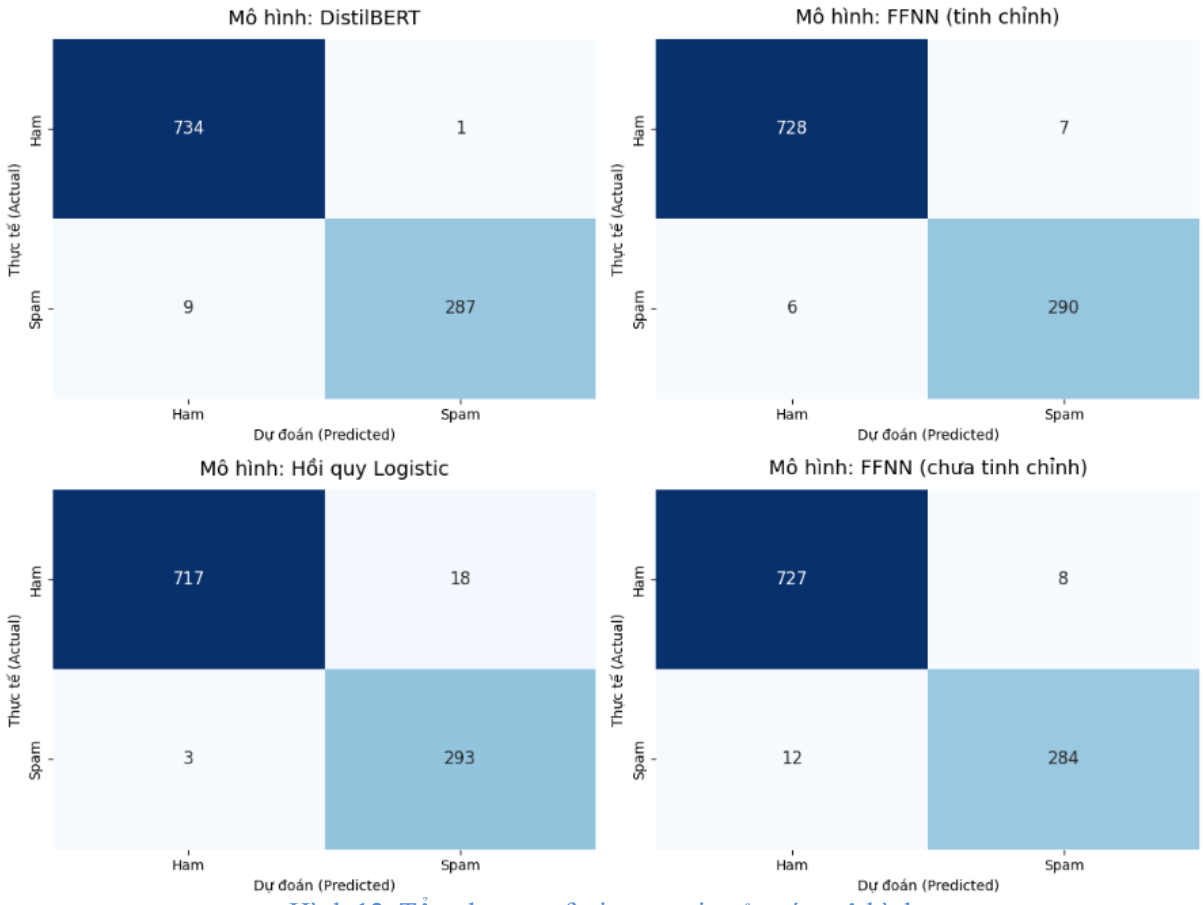
- Confusion Matrix (Ma trận nhầm lẫn): Một bảng trực quan hóa hiệu suất của mô hình, hiển thị chi tiết số lượng dự đoán đúng (True Positives - TP, True Negatives - TN) và sai (False Positives - FP, False Negatives - FN) cho từng lớp. Ma trận này cung cấp một cái nhìn sâu sắc về các loại lỗi mà mô hình đang mắc phải.

8.2 Phân tích hiệu năng mô hình

Sau khi huấn luyện và tối ưu hóa, cả ba mô hình được đánh giá trên một test set riêng biệt chưa từng được sử dụng trong quá trình huấn luyện. Kết quả được tổng hợp trong các bảng dưới đây.

Mô hình	Accuracy	Precision (Spam)	Recall (Spam)	F1-Score (Spam)
Hồi quy Logistic	0.980	0.94	0.99	0.97
FFNN (chưa tinh chỉnh)	0.981	0.97	0.96	0.97
FFNN (tinh chỉnh)	0.989	0.98	0.98	0.98
DistilBERT	0.990	1.00	0.97	0.98

Hình 11. Tổng hợp chỉ số đánh giá các mô hình



Hình 12. Tổng hợp confusion matrix của các mô hình

9. Thảo luận

Phân tích kết quả từ bảng tổng hợp đánh giá các mô hình và các ma trận nhầm lẫn cho phép đưa ra những nhận xét và đánh giá sâu sắc về hiệu suất tương đối của ba phương pháp tiếp cận.

9.1 So sánh hiệu suất:

- **Mô hình DistilBERT** thể hiện sự vượt trội rõ rệt trên tất cả các độ đo. Với

Precision đạt 0.991, nó gần như hoàn hảo trong việc xác định thư rác, có nghĩa là khi mô hình này đánh dấu một email là spam, nó gần như chắc chắn đúng. Điều này trực tiếp đáp ứng mục tiêu quan trọng nhất của bài toán là giảm thiểu False Positives. F1-Score cao nhất (0.982) cũng khẳng định hiệu suất tổng thể cân bằng và mạnh mẽ của nó. Kết quả này xác thực giả thuyết rằng các mô hình Transformer hiện đại, với khả năng hiểu ngữ cảnh sâu sắc, là lựa chọn tối ưu cho các nhiệm vụ phân loại văn bản phức tạp.

- **Mô hình FFNN với Sentence Embeddings** cũng đạt được kết quả rất ấn tượng. Precision 0.975 chỉ ra rằng nó cũng rất đáng tin cậy. Hiệu suất cao này chứng tỏ sức mạnh của việc sử dụng semantic embeddings (các biểu diễn ngữ nghĩa) được tiền huấn luyện. Ngay cả khi không cần tinh chỉnh toàn bộ một mô hình Transformer lớn, việc tận dụng các đặc trưng ngữ nghĩa của nó cũng đã mang lại một sự cải thiện đáng kể so với phương pháp cổ điển. **Sau khi tinh chỉnh** đã có sự cải thiện đáng kể và gần như bắt kịp hiệu suất của DistilBERT. Accuracy tăng lên 0.989, và các chỉ số Precision, Recall, F1-Score cho lớp Spam đều đạt 0.98. Điều này cho thấy việc cho phép mô hình ngôn ngữ tự điều chỉnh (fine-tuning) là một phương pháp cực kỳ hiệu quả để tăng cường độ chính xác.

- **Mô hình Hồi quy Logistic** đóng vai trò là một mô hình cơ sở hiệu quả để so sánh. Mặc dù có hiệu suất thấp nhất trong ba mô hình, vẫn đạt được các chỉ số hiệu suất đáng kể (F1-Score ~0.94), chỉ ra rằng rằng ngay cả một mô hình tuyến tính đơn giản kết hợp với BoW cũng có thể giải quyết bài toán này ở một mức độ hiệu quả nhất định.

9.2 Phân tích lỗi và sự đánh đổi:

- **False Positive - FP:** Đây là loại lỗi nghiêm trọng nhất.
  Phân tích confusion matrix sau khi cập nhật cho thấy:
  - DistilBERT: Chỉ có 1 email ham bị phân loại nhầm.
  - FFNN (fine-tuning): Có 7 email ham bị phân loại nhầm. Con số này đã giảm so với trước khi tinh chỉnh, cho thấy sự cải thiện.
  - Hồi quy Logistic: Có đến 18 email ham bị phân loại nhầm.

  Sự chênh lệch này một lần nữa nhấn mạnh tầm quan trọng của việc lựa chọn mô hình phù hợp để giảm thiểu rủi ro

- **Đánh đổi giữa Hiệu suất và Tài nguyên:**
  - Hồi quy Logistic: Nhanh nhất, yêu cầu ít tài nguyên nhất, nhưng rủi ro FP cao.
  - FFNN + Embeddings (fine-tuning): Cung cấp một sự cân bằng tuyệt

vời. Quá trình fine-tuning đòi hỏi nhiều tài nguyên hơn so với việc chỉ huấn luyện trên đặc trưng tĩnh, nhưng vẫn ít hơn so với DistilBERT và mang lại hiệu suất rất cao

- DistilBERT: Yêu cầu nhiều tài nguyên nhất nhưng mang lại hiệu suất và độ tin cậy cao nhất.

9.3 Khuyến nghị cuối cùng

Dựa trên tất cả các phân tích, **mô hình DistilBERT được tinh chỉnh vẫn là giải pháp được đề xuất hàng đầu**. Tuy nhiên, mô hình FFNN sau khi tinh chỉnh cũng nổi lên như một lựa chọn thay thế cực kỳ hiệu quả, gần như ngang bằng về hiệu suất nhưng có thể tiết kiệm tài nguyên hơn một chút. Cả hai mô hình học sâu đều vượt trội hoàn toàn so với phương pháp cổ điển, đáp ứng tốt mục tiêu tối đa hóa Precision để xây dựng một bộ lọc thư rác an toàn và đáng tin cậy.

10. Kết luận và Hướng phát triển

Dự án đã thực hiện thành công việc xây dựng, so sánh và đánh giá ba mô hình học máy khác nhau cho bài toán phân loại email spam và ham. Báo cáo đã trình bày một cách có hệ thống toàn bộ quy trình, từ việc phân tích bài toán, xử lý dữ liệu, thiết kế và huấn luyện mô hình, cho đến việc đánh giá hiệu suất một cách toàn diện.

Vấn đề thư rác đã được xác định không chỉ là sự phiền nhiễu mà còn là một mối đe dọa an ninh thông tin nghiêm trọng, từ đó nhấn mạnh tầm quan trọng của việc xây dựng một bộ lọc hiệu quả. Mục tiêu chính của dự án được xác định là tối đa hóa **Precision** để giảm thiểu rủi ro phân loại nhầm các email quan trọng.

Ba mô hình được triển khai đại diện cho ba cấp độ công nghệ: Hồi quy Logistic (cổ điển), mạng FFNN với Sentence Embeddings (hiện đại, hiệu quả), và DistilBERT (tiên tiến, mạnh mẽ nhất). Kết quả thực nghiệm đã khẳng định một cách rõ ràng rằng mô hình DistilBERT được tinh chỉnh mang lại hiệu suất vượt trội trên tất cả các phương diện, đặc biệt là trên độ đo Precision. Điều này chứng tỏ sức mạnh của kiến trúc Transformer trong việc hiểu các sắc thái ngữ nghĩa phức tạp của ngôn ngữ tự nhiên.

Hướng phát triển tương lai:

Mặc dù đã đạt được kết quả rất tốt, vẫn có những hướng đi tiềm năng để cải thiện và mở rộng dự án:

- Thử nghiệm các mô hình Transformer lớn hơn: Có thể thử nghiệm với các mô hình lớn hơn như BERT-base hoặc RoBERTa để xem liệu có thể đạt được sự cải thiện hơn nữa về hiệu suất hay không, dù phải đánh đổi bằng chi phí tính toán cao hơn.

- Áp dụng kỹ thuật tăng cường dữ liệu (Data Augmentation): Sử dụng các kỹ thuật như back-translation hoặc thay thế từ đồng nghĩa để tạo ra các mẫu dữ liệu huấn luyện mới, giúp mô hình trở nên mạnh mẽ và khái quát hóa tốt

hơn.

- Triển khai mô hình (Deployment): Đóng gói mô hình DistilBERT tốt nhất thành một dịch vụ API (ví dụ: sử dụng FastAPI hoặc Flask) để có thể tích hợp vào các ứng dụng thực tế và thực hiện phân loại email theo thời gian thực.

11. Phụ lục

11.1 Thuật toán Adam (Adaptive Moment Estimation)

11.1.1 Cơ sở lý thuyết và Mục tiêu

Adam (Adaptive Moment Estimation) là một thuật toán tối ưu hóa dựa trên gradient được đề xuất nhằm tích hợp các đặc tính ưu việt từ hai phương pháp tiền nhiệm: **Momentum** và **RMSprop**.

- Từ Momentum, Adam kế thừa nguyên lý tích lũy mô-men bậc nhất, tức là trung bình trượt theo cấp số nhân của các vector gradient. Cơ chế này cho phép thuật toán duy trì "đà", giúp tăng tốc quá trình hội tụ và vượt qua các điểm cực tiểu cục bộ hoặc các vùng có bề mặt hàm mất mát phẳng.

- Từ RMSprop, Adam kế thừa cơ chế thích ứng hóa tốc độ học cho từng tham số riêng biệt thông qua việc sử dụng mô-men bậc hai—trung bình trượt theo cấp số nhân của bình phương các gradient. Điều này giúp ổn định quá trình huấn luyện, đặc biệt hiệu quả đối với các bài toán có không gian tham số lớn và gradient thưa (sparse gradients).

Thông qua việc tổng hợp hai cơ chế này, Adam có khả năng tự động điều chỉnh tốc độ học và thể hiện hiệu suất hội tụ mạnh mẽ, qua đó trở thành một trong những lựa chọn nền tảng cho nhiều ứng dụng học sâu.

Nguyên lý toán học

Tại mỗi bước lặp `t`, thuật toán Adam thực hiện quy trình cập nhật tham số θ như sau:

- **Tính toán Gradient:** Xác định vector gradient gt của hàm mục tiêu L tại điểm tham số θ_{t-1}:

$$g_t = \nabla_\theta L(\theta_{t-1})$$

- **Cập nhật Ước tính Mô-men bậc nhất:** Cập nhật vector mt (ước tính trung bình của gradient) với hệ số suy giảm β_1 (thông thường ~0.9).

$$m_t = \beta_1 m_t - 1 + (1 - \beta_1) g_t$$

- **Cập nhật Ước tính Mô-men bậc hai:** Cập nhật vector v_t (ước tính phương sai không định tâm của gradient) với hệ số suy giảm β2 (thông thường ~0.999).

$$v_t = \beta_2 v_t - 1 + (1 - \beta_2) g_t^2$$

- **Bias Correction (Hiệu chỉnh Thiên vị):** Để hiệu chỉnh sự thiên vị của các ước tính mô-men vốn được khởi tạo bằng zero, thuật toán thực hiện một bước hiệu chỉnh cho m_t và v_t:

$$\widehat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

- **Cập nhật Tham số:** Thực hiện cập nhật tham số cuối cùng, trong đó tốc độ học η được điều chỉnh bởi các ước tính mô-men đã hiệu chỉnh. Hằng số ϵ (ví dụ: 10^{-8}) được đưa vào để đảm bảo tính ổn định số học.

$$\theta_t = \theta_{t-1} - \eta \frac{\widehat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

11.2 Thuật toán AdamW (Adam with Decoupled Weight Decay)

Weight Decay (tương đương về mặt toán học với điều chuẩn L2) là một kỹ thuật điều chuẩn phổ biến nhằm giảm thiểu hiện tượng overfitting thông qua việc áp dụng một thành phần phạt lên các trọng số có độ lớn cao. Trong các phương pháp triển khai Adam truyền thống, thành phần phạt này ($\lambda\theta_{t-1}$) được hợp nhất trực tiếp vào vector gradient trước khi tính toán các mô-men:

$$g_t = \nabla_\theta L(\theta_{t-1}) + \lambda\theta_{t-1}$$

Tuy nhiên, phương pháp này dẫn đến một hạn chế cố hữu. Do Adam chuẩn hóa bước cập nhật bằng ước tính mô-men bậc hai (v_t), thành phần Weight Decay cũng bị ảnh hưởng bởi quá trình chuẩn hóa này. Hệ quả là hiệu quả điều chuẩn bị suy giảm đối với các trọng số có gradient lớn trong quá khứ, làm cho cơ chế Weight Decay hoạt động không nhất quán.

11.2.1 Nguyên lý cải tiến của AdamW

AdamW, đề xuất bởi Loshchilov và Hutter, khắc phục hạn chế trên thông qua một cải tiến trực tiếp: **tách rời (decoupling)** quá trình áp dụng Weight Decay khỏi bước cập nhật gradient. Thay vì điều chỉnh gradient, AdamW áp dụng thành phần suy giảm trọng số trực tiếp trong quy tắc cập nhật tham số cuối cùng.

Công thức cập nhật của AdamW được định nghĩa lại như sau:

$$\theta_t = \theta_{t-1} - \eta\left(v\frac{\widehat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\theta_{t-1}\right)$$

Phân tích sự khác biệt:

- **Adam:** Thành phần Weight Decay được tích hợp vào gradient và do đó bị

ảnh hưởng bởi cơ chế điều chỉnh tốc độ học thích ứng.

- **AdamW:** Thành phần Weight Decay được tách rời, hoạt động độc lập và không bị ảnh hưởng bởi việc chuẩn hóa gradient, khôi phục lại cơ chế điều chuẩn L2 nguyên bản.

Cải tiến này giúp tăng cường hiệu quả điều chuẩn và cải thiện khả năng tổng quát hóa của mô hình. Do đó, AdamW thường cho thấy hiệu năng vượt trội so với Adam trong việc huấn luyện các mô hình tham số lớn như Transformer và được xem là thuật toán tối ưu hóa tiêu chuẩn cho các tác vụ này.